SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

2026 **BC. Marcel Soták**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav informatiky a matematiky (FEI)
Školiteľ:	Ing. Maroš Čavojský, PhD.

2026 **BC. Marcel Soták**

Slovenská technická univerzita v Bratislave
Ústav jadrového a fyzikálneho inžinierstva

Fakulta elektrotechniky a informatiky
Akademický rok: 2024/2025
Evidenčné číslo: FEI-XXXXX-YYYYYY

ZADANIE BAKALÁRSKEJ PRÁCE

Študent:	**Meno Priezvisko**
ID študenta:	012345
Študijný program:	jadrové a fyzikálne inžinierstvo
Študijný odbor:	elektrotechnika
Vedúci práce:	tituly, Meno Priezvisko, tituly
Vedúci pracoviska:	tituly Meno Priezvisko, tituly
Miesto vypracovania:	Ústav jadrového a fyzikálneho inžinierstva

Názov práce: **Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme LaTeX**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

1. Vytvorte šablónu záverečnej práce pre študentov bakalárskeho a inžinierskeho štúdia na FEI STU v Bratislave.
2. Napíšte návod na písanie záverečnej práce s použitím vytvorenej šablóny.
3. Vytvorte prehľad najpoužívanejších typov citovaných zdrojov a zdokumentujte ich tak, aby boli v súlade s normou ISO 690.

Zoznam odbornej literatúry:

1. Zákon č. 131/2002 Z. z. z 21. februára 2002 o vysokých školách a o zmene a doplnení niektorých zákonov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2002/131/ZZ_2002_131_20240901.pdf
2. Vyhláška č. 233/2011 Z. z. Ministerstva školstva, vedy, výskumu a športu Slovenskej republiky z 1. júla 2011, ktorou sa vykonávajú niektoré ustanovenia zákona č. 131/2002 Z. z. o vysokých školách a o zmene a doplnení niektorých zákonov v znení neskorších predpisov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2011/233/ZZ_2011_233_20201015.pdf

3. Metodické usmernenie Ministerstva školstva, vedy, výskumu a športu SR č. 56/2011 o náležitostiach záverečných prác, ich bibliografickej registrácii, kontrole originality, uchovávaní a sprístupňovaní č. 2011-11513/31015:4-071 [online]. [cit. 2024-09-24]. Dostupné na http://www.minedu.sk/metodicke-usmernenie-c-562011-o-nalezitostiach-zaverecnych-prac-ich-bibliografickej-registracii-uchovavani-a-spristupnovani/.
4. STN ISO 214: 1998. Dokumentácia. Abstrakty (referáty) pre publikácie a dokumentáciu. Bratislava: Slovenský ústav technickej normalizácie. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
5. STN ISO 690: 2022. Dokumentácia – Bibliografické odkazy – Obsah, forma a štruktúra. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
6. STN 01 6910: 2023. Pravidlá písania a úpravy písomností. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
7. STN ISO 2145: 1997. Dokumentácia. Číslovanie oddielov a pododdielov písaných dokumentov. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
8. STN ISO EN 80 000-1: 2022. Veličiny a jednotky. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
9. LICHNEROVÁ, L. a B. BELLÉROVÁ. Nové pravidlá citovania podľa ISO 690 z roku 2021. *ITlib. Informačné technológie a knižnice,* 3–4/2023, 10 – 22. [cit. 2024-09-06]. Dostupné na http://doi.org/10.52036/1335793X.2023.3-4.10-22.
10. HOFTICH, M. ISO 690 biblatex style [online]. [cit. 2024-09-30]. Dostupné na https://mirrors.ibiblio.org/CTAN/macros/latex/contrib/biblatex-contrib/biblatex--iso690/biblatex-iso690.pdf.

Termín odovzdania bakalárskej práce:	31. 05. 2025
Dátum schválenia zadania bakalárskej práce:	28. 02. 2025
Zadanie bakalárskej práce schválil:	tituly Meno Priezvisko, tituly – garant študijného programu

Poďakovanie

Týmto by som sa chcel úprimne poďakovať vedúcemu mojej diplomovej práce Ing. Marošovi Čavojskému, PhD. za jeho cenné rady, pomoc a vedenie počas písania tejto práce. Moje poďakovanie patrí taktiež mojej rodine za správnu výchovu, potrebunú k dopraqcovaniu až sem,mojej priateľke za psychickú podporu a poskytnutie príjemného prostredia pri písaní práce, ako aj Ministerstvu školstva za bezplatnú možnosť študovať na tejto fakulte.

Abstrakt

Táto diplomová práca sa zameriava na vývoj aplikácie, ktorá používateľom umožňuje jednoduché sociálne prepájanie prostredníctvom organizovania a vyhľadávania podujatí na základe ich polohy. Aplikácia podporuje vytváranie menších neformálnych stretnutí jednotlivcov so spoločnými záujmami, ako aj organizovanie väčších podujatí zo strany firiem a organizácií.

Cieľom práce bolo vytvoriť používateľsky prívetivú aplikáciu využívajúcu moderné postupy a technológie, ktorá zjednodušuje interakciu medzi používateľmi so spoločnými záujmami a podporuje živé stretnutia v reálnom čase.

Systém je navrhnutý ako Android aplikácia využívajúca GPS technológiu na zistenie polohy používateľa a poskytuje možnosti pre vytváranie a správu udalostí, pozývanie účastníkov a správu notifikácií. Aplikácia funguje aj na pozadí a je schopná posielať notifikácie. Využíva Firebase databázu pre ukladanie a synchronizáciu dát v reálnom čase, ako aj vlastný odporúčací systém pre navrhovanie udalostí na základe histórie používateľa a jeho záujmov. Aplikácia má taktiež implementovaný systém na detekciu nevhodných výrazov v názvoch a popisoch podujatí.

Výsledná aplikácia bola otestovaná na reálnych zariadeniach a hodnotená z hľadiska funkčnosti a použiteľnosti.

Kľúčové slová

mobilná aplikácia, správa udalostí, geolokácia, Firebase, Android, Kotlin

Abstract

The manual for students of the Faculty of Electrical Engineering and Information Technology at Slovak University of Technology in Bratislava provides advice and guidelines on how to approach the formal aspects of writing a final thesis for university studies. The document can also serve as a template for the thesis in the LaTeX typesetting system. It covers in detail the rules for typesetting mathematical equations, numbering floating objects, and referencing them. It also thoroughly addresses the method of citing external literary sources.

Keywords

mobile application, event management, geolocation, Firebase, Android, Kotlin

Obsah

Zoznam značiek a skratiek

$\vec{a}, \boldsymbol{a}$ – zrýchlenie (m/s^2)

a – veľkosť zrýchlenia (m/s^2), mriežková konštanta (nm)

AC – striedavý prúd, z angl. *Accelerating Current*

AFM – mikroskop atómových síl, z angl. *Atomic Force Microscope*

AI – umelá inteligencia, z angl. *Artificial Intelligence*

ε_0, ε_0 – elektrická konštanta ($\varepsilon_0 = 8{,}85418782\,.\,10^{-12}\,\mathrm{m^{-3}kg^{-1}s^4A^2}$)

GAI – generatívna umelá inteligencia

λ – vlnová dĺžka svetla (nm)

LED – svetlo emitujúca dióda, z angl. *Light-Emitting Diode*

MFA – viacfaktorové overenie, z angl. *Multi-Factor Authentication*

LLM – veľký jazykový model, z angl. *Large Language Model*

$\vec{F}, \boldsymbol{F}, F$ – vektor sily, veľkosť sily (N)

q – elektrický náboj (C)

$\boldsymbol{r}$, r – polohový vektor, veľkosť polohového vektora (m)

SEM – skenovací elektrónový mikroskop

TEM – transmisný elektrónový mikroskop

U – elektrické napätie (V)

Zoznam algoritmov

1 Vypočítaj $y = x^n$ 96

Zoznam výpisov kódov

1 Registrácia používateľa v AuthService 51
2 Prihlásenie cez Google v AuthService 51
3 Prihlásenie pomocou e-mailu a hesla 52
4 Validácia hesla v RegisterViewModel 52
5 Inicializácia MapView v prostredí Compose 53
6 Computed property activeEvents v AppViewModel 53
7 Uloženie eventu do Firestore 55
8 Stavové premenné v detaile eventu 55
9 Prihlásenie na event 56
10 Odhlásenie z eventu 56
11 Dátová trieda FilterCriteria 56
12 Príklad podmienky vo EventFilterService 57
13 Haversine vzorec 57
14 Aplikovanie filtrov na mapu 58
15 Polia hodnotenia v dátovom modeli eventu 58
16 Implementácia metódy rateEvent 58
17 Logika check-inu v AppViewModel 60
18 Polia no-show a soft ban v UserProfile 61
19 Implementácia recordNoShow 62
20 Konštanty systému nahlasovania 63
21 Implementácia metódy reportEvent 64
22 Automatické zablokovanie používateľa 64
23 Implementácia real-time listenera vo ViewModeli 66
24 Určenie stavu priateľstva 66
25 Načítanie priateľstiev z Firestore 67
26 Odoslanie žiadosti o priateľstvo 67
27 Prijatie alebo odmietnutie žiadosti 68
28 Deterministické generovanie ID konverzácie 68
29 Predvolené váhy odporúčacieho systému 70
30 Normalizácia váh 70
31 Výpočet skóre hlavnej kategórie 71
32 Výpočet skóre vzdialenosti 72
33 Výpočet skóre ratingu 72
34 Výpočet skóre popularity 73
35 Výpočet bonusu za obľúbenú kategóriu 73

36 Odoslanie pozvánky s kontrolou duplicity 74
37 Reakcia na pozvánku 75
38 Plánovanie notifikácie pomocou AlarmManager 76
39 Zrušenie naplánovanej notifikácie 77
40 Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting` 97

Úvod

Digitalizácia priniesla množstvo výhod, od online nakupovania až po digitálnu komunikáciu. Popri týchto výhodách však možno identifikovať aj negatívne vplyvy. Nadmerná konzumácia digitálneho obsahu môže viesť k pasivite, zhoršeniu duševného zdravia a zníženiu celkovej produktivity. Hoci sa na prvý pohľad zdá, že tieto aktivity ponúkajú zábavu a únik, v skutočnosti môžu mať opačný efekt. Človek sa stáva závislejším od technológií, čo často vedie k potláčaniu vlastných záujmov, záľub a rozvíjania talentov, ktoré by inak mohol v reálnom svete objavovať.

Odpoveďou na tieto výzvy môže byť mobilná aplikácia, ktorá uľahčuje vyhľadávanie komunity s podobnými záujmami a objavovanie nových voľnočasových aktivít na základe vizuálnej a lokalizačnej preferencie. Hlavnou motiváciou pre vývoj tejto aplikácie je snaha vytvoriť platformu, ktorá motivuje používateľov k opusteniu digitálneho priestoru v prospech reálnych spoločenských udalostí a osobného rozvoja.

Cieľom tejto diplomovej práce je vytvoriť mobilnú aplikáciu pre systém Android, ktorá spája ľudí, pomáha organizátorom jednoducho vytvárať vlastné podujatia a pomáha používateľom nájsť zábavu, nové záujmy alebo len tak tráviť čas s priateľmi. Pri tejto práci využijeme množstvo postupov a technológií, ktoré nám umožnia vytvoriť funkčnú a používateľsky príjemnú aplikáciu.

V práci sa budeme venovať návrhu a implementácii tejto aplikácie. Využijeme databázy na ukladanie informácií o používateľoch a podujatiach a navrhneme odporúčací systém vzhľadom na používateľovu históriu. Taktiež navrhneme systém, ako zamedziť zneužívaniu aplikácie a ako zabezpečiť kontrolu nevhodného obsahu. Aplikácia bude mať jednoduché a intuitívne rozhranie, ktoré umožní používateľom ľahko sa orientovať a využívať všetky funkcie. Nakoniec budeme aplikáciu testovať a získavať spätnú väzbu od používateľov, aby sme ju mohli vylepšiť a prispôsobiť ich potrebám. Verím, že táto aplikácia bude užitočná pre mnoho ľudí, pomôže im nájsť nové záujmy a že jej finálna verzia prinesie do tohto sveta hodnotu.

1 Základné pojmy

V tejto kapitole sa zameriame na všeobecné pojmy a technológie, ktoré sú kľúčové pre správne pochopenie problematiky vývoja mobilných aplikácií a implementácie odporúčacích systémov.

1.1 Správa identít

1.1.1 Autentifikácia

Autentifikácia je proces overenia používateľa teda zistenie, o koho ide. Môžeme overovať podľa kombinácie mena a hesla, biometrie (napr. otlačku prsta alebo rozpoznania hlasu), alebo iným zariadením

1.1.2 Faktory a metódy autentifikácie

Spôsoby overenia používateľa delíme do troch základných kategórií podľa toho, na čom sú založené:

- **Znalostný faktor:** Niečo, čo používateľ pozná
- **Faktor vlastníctva:** Niečo, čo používateľ má
- **Dedičný faktor:** Niečo, čím používateľ je

Pre dosiahnutie vysokej bezpečnosti sa odporúča é overenie (angl. *Multi-Factor Authentication*, MFA), ktoré kombinuje aspoň dva z vyššie uvedených faktorov. V mobilných zariadeniach sa čoraz častejšie využíva aj tzv. kontinuálne overovanie, ktoré neustále monitoruje identitu používateľa na základe biometrie správania (napr. štýl písania alebo dynamika dotyku) [1].

1.1.3 Autorizácia

Autorizácia je proces overenia oprávnení pre danú úlohu, teda či autentifikovaný používateľ (systém) má oprávnenie vykonať úlohu, o ktorú žiada (napr. vstúpiť do systému alebo vykonať platobnú operáciu).

1.1.4 Mechanizmus trvalého prihlásenia (Token-based Authentication)

V moderných mobilných aplikáciách je žiaduce, aby používateľ nebol nútený zadávať prihlasovacie údaje pri každom otvorení aplikácie. Tento proces sa nazýva perzistentná autentifikácia a je realizovaný pomocou systému dvoch typov tokenov (bezpečnostných kľúčov):

- **Access Token (Prístupový kľúč):** Krátkodobý token ktorý mobilná aplikácia pripája ku každej požiadavke na server. Slúži ako dôkaz, že používateľ je momentálne overený.
- **Refresh Token (Obnovovací kľúč):** Dlhodobý token uložený v šifrovanej pamäti zariadenia. Jeho jedinou úlohou je vyžiadať si nový *Access Token* v prípade, že pôvodný vypršal.

Tento cyklus prebieha na pozadí bez zásahu používateľa (tzv. *Silent Re-authentication*). Ak je *Refresh Token* stále platný, aplikácia automaticky obnoví reláciu, čím zabezpečí plynulý používateľský zážitok pri zachovaní vysokej bezpečnosti, keďže samotné heslo sa sieťou prenáša iba raz – pri prvotnom prihlásení [2].

1.2 Technológia QR kódov a ich štruktúra

QR kód (*Quick Response Code*) predstavuje dvojrozmerný čiarový kód, ktorý bol navrhnutý pre vysokorýchlostné čítanie a veľkú kapacitu dát v porovnaní s tradičnými 1D kódmi. Jeho štruktúra je definovaná niekoľkými kľúčovými prvkami, ktoré umožňujú spoľahlivú detekciu aj pri čiastočnom poškodení alebo zlej kvalite obrazu:

- **Vyhľadávacie vzory (Finder Patterns):** Tri identické štvorcové štruktúry v rohoch kódu, ktoré umožňujú skeneru identifikovať polohu a orientáciu kódu v 360-stupňovom rozsahu.
- **Korekcia chýb (Error Correction):** Využitie algoritmu *Reed-Solomon*, ktorý umožňuje obnoviť dáta aj v prípade, že je kód znečistený alebo poškodený (v závislosti od úrovne až do 30
- **Dátová kapacita:** Na rozdiel od lineárnych kódov dokáže QR kód uchovávať tisíce alfanumerických znakov, čo je kľúčové pre kódovanie komplexných URL adries alebo unikátnych identifikátorov podujatí.

[3]

1.3 Prístupy k vývoju mobilných aplikácií

Podľa spoločnosti IBM možno proces vývoja mobilných aplikácií definovať ako tvorbu softvéru navrhnutého špecificky pre mobilné koncové zariadenia, pričom kľúčovým rozhodnutím je technológia implementácie. V súčasnosti sa vývojári rozhodujú primárne medzi natívnym a multiplatformovým prístupom [**ibm_mobile_app_dev**]:

- **Natívne aplikácie:** Sú vyvíjané priamo pre konkrétnu platformu (Android, iOS) pomocou dedikovaných nástrojov. Ponúkajú najvyšší výkon a neobmedzený prístup k hardvéru.
- **Webové mobilné aplikácie:** Ide o responzívne webové stránky využívajúce technológie HTML5, CSS a JavaScript. Modernou formou sú tzv. *Progressive Web Apps* (PWA).
- **Hybridné aplikácie:** Kombinujú predchádzajúce prístupy. Jadro aplikácie je písané vo webových technológiách, ktoré sú následne "zabalené"do natívneho kontajnera.

[4]

1.4 Geolokačné technológie a ich využitie

Geolokácia je proces identifikácie geografickej polohy zariadenia. Podľa analýzy spoločnosti Sensolus [**sensolus_geoloc**] sa v moderných systémoch využívajú štyri hlavné technológie, ktoré sa líšia presnosťou a energetickou náročnosťou:

- **GPS (Global Positioning System):** Využíva satelitné signály. Je najpresnejší v exteriéri (na metre), ale má vysokú spotrebu batérie a nefunguje vo vnútri budov.
- **Wi-Fi Sniffing:** Určuje polohu skenovaním okolitých Wi-Fi sietí. Je ideálny pre mestské prostredie a prechod medzi budovami, pričom spotrebúva menej energie ako GPS.
- **BLE (Bluetooth Low Energy):** Využíva malé vysielače (beacons). Poskytuje extrémnu presnosť v interiéroch (napr. sklady), ale vyžaduje inštaláciu fyzických zariadení v priestore.
- **LPWAN Geolocation:** Využíva nízkoenergetické siete (napr. Sigfox) na trianguláciu signálu. Má najdlhšiu výdrž batérie (roky), ale nižšiu presnosť (stovky metrov).

1.4.1 Operačná efektivita a hybridný prístup

Výber správnej technológie priamo ovplyvňuje prevádzkovú efektivitu systému. V mobilnom vývoji sa najčastejšie využíva tzv. **hybridná geolokácia**, ktorá inteligentne prepína medzi GPS a Wi-Fi podľa dostupnosti signálu a stavu batérie. To umožňuje aplikáciám efektívne sledovať polohu používateľa bez nadmerného zaťaženia hardvéru. (Zdroj: `https://www.sensolus.com/knowledge-hub/behind-the-tech/four-geolo`

cation-technologies-compared-how-can-they-improve-your-operational-efficiency/)

1.4.2 Matematické metódy určovania polohy

Na výpočet presnej polohy zariadenia zo získaných signálov (rádiových, satelitných či Bluetooth) sa v informatike využívajú nasledujúce geometrické a fyzikálne metódy:

- **TOA (Time of Arrival):** Metóda meria absolútny čas, za ktorý signál prejde od vysielača k prijímaču. Na určenie polohy v 2D priestore sú potrebné aspoň tri vysielače (trilaterácia).
- **TDOA (Time Difference of Arrival):** Nameria sa rozdiel v čase príchodu signálov z viacerých vysielačov. Je to praktickejšie ako TOA, pretože nevyžaduje dokonalú synchronizáciu hodín medzi vysielačom a mobilným zariadením.
- **AOA (Angle of Arrival):** Táto metóda neurčuje vzdialenosť, ale **uhol**, pod ktorým signál prichádza k anténe. Kombináciou uhlov z dvoch staníc je možné určiť presný bod polohy (triangulácia).
- **RSS (Received Signal Strength):** Odhaduje vzdialenosť na základe útlmu sily signálu. Čím je signál slabší, tým je zariadenie pravdepodobne ďalej od vysielača (často využívané pri Wi-Fi a Bluetooth).

Tieto metódy sú matematickým základom pre fungovanie GPS (prevažne TOA/TDOA) aj moderných systémov pre lokalizáciu v interiéroch.

1.5 Definícia a význam odporúčacích systémov

S prudkým rozvojom internetových technológií a inteligentných mobilných zariadení čelí moderný používateľ fenoménu informačného preťaženia. V prostredí s takmer neobmedzeným množstvom digitálneho obsahu a služieb sa odporúčacie systémy (angl. *Recommender Systems*) stali kľúčovým nástrojom na personalizovanú filtráciu informácií.

Ich primárnou úlohou je predpovedať preferencie používateľa a zredukovať rozsiahly priestor dostupných položiek na úzky výber relevantných možností, čím sa zvyšuje celková spokojnosť a efektivita interakcie s platformou. Súčasný výskum sa čoraz viac orientuje na spracovanie implicitných dát. Ide o behaviorálne vzorce, ako sú kliknutia (*click-stream*), história navštívených miest či čas strávený interakciou s konkrétnym objektom [**electronics11010141**].

1.5.1 Obsahové filtrovanie (Content-Based Filtering)

Tento prístup je založený na analýze špecifických atribútov položiek. Systém identifikuje vlastnosti objektov (napr. textové informácie, kategórie alebo tagy), o ktoré používateľ v minulosti prejavil záujem, a na ich základe generuje odporúčania pre podobné položky [**electronics11010141**].

- **Princíp:** Analýza atribútov a metadát položiek (napr. žáner, kategória) s cieľom nájsť podobnosť s preferenciami používateľa.
- **Využitie:** Efektívne najmä pri službách pracujúcich s textovými dátami, hudbou alebo e-commerce [**electronics11010141**].
- **Limity:** Nízka miera diverzity (angl. *serendipity*) – používateľovi nie sú ponúkané nové, neobjavené kategórie obsahu, čo vedie k efektu informačnej bubliny.

1.5.2 Kolaboratívne filtrovanie (Collaborative Filtering)

Kolaboratívne filtrovanie predpovedá vkus používateľa na základe analýzy správania a hodnotení celej komunity používateľov (tzv. „wisdom of the crowd“).

- **Memory-based:** Hľadá podobnosť priamo medzi používateľmi (*User-based*) alebo položkami (*Item-based*) pomocou algoritmov ako KNN alebo korelačných koeficientov.
- **Model-based:** Využíva pokročilé štatistické metódy a techniky strojového učenia, ako sú zhlukovanie (*Clustering*), *Singular Value Decomposition* (SVD) alebo *Principal Component Analysis* (PCA).
- **Hlavné nedostatky:** Zdroj [**electronics11010141**] definuje tri kritické obmedzenia:
  - *Cold Start:* Problém s odporúčaním položiek pre nových používateľov bez histórie interakcií.
  - *Sparsity (Riedkosť dát):* Situácia, keď je v systéme príliš málo hodnotení na to, aby bolo možné spoľahlivo vypočítať podobnosť.
  - *Gray Sheep (Sivé ovce):* Používatelia s veľmi unikátnym vkusom, ktorí nevykazujú podobnosť so žiadnou väčšou skupinou používateľov, čo sťažuje generovanie odporúčaní.

1.5.3 Hybridné systémy

Hybridné modely kombinujú viaceré techniky s cieľom eliminovať slabiny jednotlivých prístupov, najmä problém riedkosti dát (*sparsity*) a studeného štartu. Podľa Burkeho klasifikácie [**electronics11010141**] existuje sedem základných metód hybridizácie, ktoré sú uvedené v tabuľke **??**. Hlavným cieľom väčšiny štúdií zaoberajúcich sa hybridnými

Tabuľka 1: Typy hybridných odporúčacích systémov podľa Burkeho [**electronics11010141**]

Metóda	**Princíp fungovania**
Weighted	Skóre z rôznych techník sa kombinuje pomocou numerických váh.
Switching	Systém prepína medzi technikami v závislosti od kontextu.
Mixed	Odporúčania z viacerých zdrojov sú zobrazené súčasne.
Feature Combination	Atribúty z jedného zdroja slúžia ako vstupné črty pre druhý model.
Feature Augmentation	Výstup jednej techniky sa stáva parametrom pre ďalšie spracovanie.
Cascade	Postupné zjemňovanie poradia (jeden systém vytvorí rebríček, druhý ho upresní).
Meta-level	Celý model vygenerovaný jednou technikou slúži ako vstup pre druhú.

modelmi po roku 2010 je integrácia tzv. vedľajších informácií (*side information*), ako sú sociálne väzby medzi používateľmi alebo kontextové dáta, ktoré dopĺňajú chýbajúce explicitné hodnotenia. Týmto spôsobom hybridné systémy dosahujú vyššiu presnosť a robustnosť v porovnaní so samostatnými modelmi.

Kritérium	Collaborative	Content	Hybrid	Knowledge
Cold Start (používateľ)	Zlé	Zlé	Stredné	Dobré
Cold Start (položka)	Zlé	Dobré	Stredné	Dobré
Škálovateľnosť	Zlé	Dobré	Stredné	Dobré
Diverzita	Dobré	Zlé	Dobré	Stredné
Potreba metadát	Nie	Áno	Čiastočne	Áno
Vysvetliteľnosť	Stredné	Dobré	Stredné	Dobré

Tabuľka 2: Porovnanie typov odporúčacích systémov

Porovnanie systémov

1.6 Techniky spracovania dát v odporúčacích systémoch

Pre efektívne fungovanie vyššie spomínaných modelov sa využívajú špecifické techniky dolovania dát (Data Mining):

1. **Text Mining:** Analýza neštruktúrovaných textov pomocou sémantickej analýzy a váhovania pojmov (napr. TF-IDF). Slúži na pochopenie obsahu podujatí a preferencií používateľa.

2. **K-Nearest Neighbor (KNN):** Algoritmus slúžiaci na klasifikáciu položiek alebo používateľov na základe ich blízkosti v metrickom priestore (geografickom alebo vkusovom).

3. **Clustering (Zhlukovanie):** Automatické rozdeľovanie dát do skupín so spoločnými črtami. Využíva sa najmä pri segmentácii používateľov alebo kategorizácii lokalít.

4. **Matrix Factorization:** Pokročilá technika rozkladu matíc, ktorá transformuje interakcie používateľov do priestoru latentných faktorov, čím rieši problém riedkosti dát (Sparsity).

5. **Neural Networks:** Modely hlbokého učenia schopné modelovať nelineárne vzťahy a spracovávať komplexné dáta zo sociálnych sietí či senzorov mobilných zariadení.

1.7 Architektonické vzory a princípy návrhu

Úspešná mobilná aplikácia musí byť postavená na robustnej architektúre, ktorá zaručuje jej škálovateľnosť a udržateľnosť v ekosystéme rôznych zariadení (telefóny, tablety, ChromeOS či skladacie zariadenia). Podľa oficiálnej dokumentácie spoločnosti Google je

základným cieľom architektúry definovať hranice medzi jednotlivými časťami aplikácie a jasne určiť ich zodpovednosť.

1.8 Základné princípy moderného návrhu

Pred definovaním konkrétnych vzorov je potrebné určiť pravidlá, ktoré musí moderná architektúra spĺňať pre zachovanie stability v prostredí s obmedzenými systémovými prostriedkami:

- **Separation of concerns (Oddelenie zodpovedností):** Kľúčový princíp, ktorý zakazuje sústredenie kódu v triedach *Activity* alebo *Fragment*. Tieto komponenty slúžia primárne ako hostitelia používateľského rozhrania, sú krátkodobé (*ephemeral*) a operačný systém ich môže kedykoľvek zničiť z dôvodu nedostatku pamäte alebo zmeny konfigurácie.
- **Drive UI from data models (Riadenie UI dátovými modelmi):** Používateľské rozhranie by malo byť postavené na dátových modeloch, ktoré sú nezávislé od životného cyklu UI prvkov. Perzistentné modely zabezpečujú, že aplikácia nestratí dáta pri reštarte procesu a zostáva funkčná aj pri nestabilnom pripojení k sieti.
- **Single Source of Truth (SSOT - Jednotný zdroj pravdy):** Každý dátový typ má priradeného svojho vlastníka, ktorý ako jediný môže dáta modifikovať. Zmeny sa následne propagujú prostredníctvom imutabilných typov, čo znižuje výskyt chýb a uľahčuje ladenie systému.
- **Unidirectional Data Flow (UDF - Jednosmerný tok dát):** Stav aplikácie prúdi smerom nadol (od SSOT k UI elementom) a udalosti/akcie prúdia smerom nahor. Tento vzor zaručuje konzistenciu dát v celom systéme.

1.9 Porovnanie architektonických vzorov

V histórii vývoja platformy Android sa vyvinulo niekoľko prístupov k organizácii kódu, ktoré reagovali na narastajúcu komplexnosť aplikácií:

1. **MVC (Model-View-Controller):** Tradičný model, kde Controller riadi tok dát. V Androide tento vzor zlyhával, pretože triedy *Activity* preberali úlohu controllera aj view súčasne, čo viedlo k vzniku ťažko udržiavateľného kódu.
2. **MVP (Model-View-Presenter):** Oddelil zodpovednosť medzi View a Presenter. Problémom zostala vysoká previazanosť a nutnosť manuálneho ošetrovania životného cyklu, čo spôsobovalo úniky pamäte (*memory leaks*).

3. **MVVM (Model-View-ViewModel):** Súčasný štandard. *ViewModel* uchováva stav aplikácie nezávisle od UI komponentov, čím rieši problém s reštartovaním aktivít a stratu dát pri zmene orientácie zariadenia.

2 Súčasný stav riešenej problematiky

Na trhu existuje množstvo platforiem zameraných na organizáciu a objavovanie podujatí. Dominantným hráčom v tejto oblasti je služba **Facebook Events**, vyvíjaná spoločnosťou Meta. Vďaka zázemiu veľkej technologickej spoločnosti ide o komplexný systém, ktorý pokrýva široké spektrum potrieb používateľov.

2.1 Facebook Events

Facebook Events umožňuje používateľom vytvárať vyhradené stránky pre konkrétne príležitosti a zasielať verejné alebo súkromné pozvánky. Pre online firmy predstavujú tieto podujatia efektívny nástroj na propagáciu predajov, osláv alebo dôležitých míľnikov. Systém zahŕňa históriu podujatí, možnosť označovania obľúbených aktivít a pokročilé filtrovanie.

- **Výhody:** Obrovská používateľská základňa, vysoká miera integrácie so sociálnou sieťou, prepracované algoritmy na predikciu záujmov a množstvo funkcií dostupných bez priamych poplatkov za vytvorenie podujatia.
- **Nevýhody a obmedzenia:**
  - **Informačné zahltenie a notifikačná únava:** Rozhranie je preplnené reklamami. Nadmerné množstvo systémových upozornení (*Notification Fatigue*) často vedie k tomu, že používatelia dôležité pozvánky na podujatia prehliadnu alebo ignorujú.
  - **Nízka miera kontroly nad dosahom:** Organizátor nemá záruku, že sa informácia o podujatí dostane ku všetkým sledovateľom. Algoritmus uprednostňuje platenú reklamu a interaktívny obsah, čo znevýhodňuje menšie lokálne a komunitné aktivity.
  - **Uzavretosť ekosystému:** Plné využitie funkcií vyžaduje existenciu Facebook profilu, čo odrádza časť modernej populácie (najmä generáciu Z).
  - **Obmedzená vizualizácia:** Mapa podujatí nie je primárnym ovládacím prvkom a je dostupná až po manuálnom rozkliknutí, čo znižuje intuitívnosť pri hľadaní aktivít v okolí.
  - **Absencia priamej komunikácie:** Diskusia v rámci podujatia prebieha formou príspevkov na nástenke, chýba však funkcia reálneho skupinového chatu pre účastníkov v reálnom čase.

Zdroje:
`https://www.facebook.com/events/`
`https://hub.theeventplannerexpo.com/social-media-marketing/the-pros-and-cons-of-facebook-events-should-you-even-bother` https://hub.theeventplannerexpo.com/social-media-marketing/the-pros-and-cons-of-facebook-events-should-you-even-bother
Zdroje:
`https://www.facebook.com/events/`
`https://www.bigcommerce.com/glossary/what-are-facebook-events/`

2.2 Meetup

Platforma **Meetup** je celosvetovo známa platforma sa primárne zameriava na budovanie dlhodobých komunít a záujmových skupín (*Groups*). Na rozdiel od Facebooku, kde sú podujatia často jednorazové, Meetup motivuje používateľov k pravidelnému stretávaniu sa na základe spoločných koníčkov, ako sú technológie, šport či jazykové vzdelávanie.

V porovnaní s Facebook Events ponúka Meetup detailnejšie komunitné funkcie, avšak väčšina z nich je podmienená plateným členstvom:

- **Rozšírená interakcia:** Platforma umožňuje prehliadať zoznamy účastníkov, pristupovať k ich profilom a využívať pokročilé systémy komentárov a správ, čo uľahčuje nadväzovanie kontaktov ešte pred samotným podujatím.
- **Personalizácia (*For You*):** Algoritmus navrhuje podujatia a skupiny na základe histórie aktivít a preferencií používateľa, pričom prioritizuje lokálne komunity.
- **Platený model (Monetizácia):** Najväčšou nevýhodou platformy je, že kľúčové funkcie pre organizátorov (vytváranie podujatí, správa členov) aj pokročilé funkcie pre účastníkov sú spoplatnené. V rámci prémiového balíka získavajú používatelia:
  - prístup k úplným zoznamom a profilom účastníkov,
  - prioritný prístup k podujatiam a čakacím listinám (*Waitlists*),
  - neobmedzené zasielanie správ ostatným členom,
  - rozhranie bez reklám a včasné upozornenia na nové udalosti.
- **Problém s účasťou:** Podobne ako pri iných bezplatných registráciách, aj Meetup čelí vysokej miere tzv. *no-shows* (používatelia, ktorí potvrdia účasť, ale neprirodzene absentujú), čo sťažuje plánovanie kapacity pre organizátorov.

Kritické zhodnotenie: Hoci Meetup poskytuje kvalitnejšie nástroje na sieťovanie (*networking*) než Facebook, jeho uzavretosť za platobnú bránu a slabšia SEO viditeľnosť pre nečlenov otvárajú priestor pre vývoj prístupnejších, bezplatných alternatív.

- **Problém „No-shows“ (Neprihlásení účastníci):** Štatistiky z praxe ukazujú, že v priemere až 50 % používateľov, ktorí potvrdia účasť (*RSVP*), na udalosť nakoniec nepríde. Tento jav je spôsobený nízkou bariérou záväzku, najmä pri bezplatných podujatiach.

- **Vysoké prevádzkové náklady:** Pre organizátorov predstavuje Meetup finančnú záťaž (poplatky od cca 25 $ mesačne), pričom základné balíky výrazne obmedzujú možnosti brandingu, vkladanie multimédií (video, vlastné logá) či prispôsobenie dizajnu skupiny.

- **Geografické obmedzenia vyhľadávania:** Systém je primárne postavený na vyhľadávaní podľa PSČ alebo mesta. To drasticky limituje *SEO* dosah udalostí, ktoré sú organizované online alebo cília na globálne publikum, nakoľko používatelia nevidia relevantné výsledky mimo svojho bezprostredného okolia.

- **Pasivita a „Ghost groups“:** Na platforme sa nachádza veľké množstvo neaktívnych skupín, ktoré slúžia len na zber e-mailových adries (*lead generation*) bez reálnej snahy organizovať stretnutia.

- **Kvalita a bezpečnosť lokalít:** Mnohé stretnutia sa odohrávajú v baroch alebo reštauráciách, čo prináša problémy s hlukom, nevhodným osvetlením alebo dokonca neprofesionálnym správaním účastníkov (alkohol). Verejné inštitúcie (knižnice) sú často pre neformálne skupiny nedostupné.

- **Chýbajúca automatizácia časových pásiem:** Meetup nedostatočne rieši automatický prepočet časových pásiem pri medzinárodných online podujatiach, čo núti organizátorov k manuálnym opravám.

Zhrnutie pre návrh novej aplikácie:
Tieto zistenia naznačujú, že moderná alternatíva by sa mala zamerať na: 1. **Zvýšenie záväznosti účastníkov** (napr. cez integrovaný chat alebo potvrdzovací systém krátko pred akciou). 2. **Transparentnú mapu bez hraníc miest**, ktorá umožní objavovať udalosti organicky scrollovaním. 3. **Natívnu podporu pre online podujatia** s automatickou prácou s časom a SEO optimalizáciou.

2.3 AllEvents.in

AllEvents sa prezentuje ako globálna platforma na objavovanie udalostí, ktorá pokrýva viac než 40 000 miest po celom svete. Platforma sa zameriava na agregáciu širokého spektra podujatí – od komunitných stretnutí až po veľké komerčné koncerty a festivaly.

- **Deklarované funkcie:** Personalizované odporúčania na základe záujmov, sledovanie aktivít priateľov, bezpapierová registrácia (*check-in*) a priama rezervácia vstupeniek v rámci aplikácie.
- **Kategorizácia a distribúcia:** Platforma vyniká prehľadným členením do kategórií, možnosťou exportu do kalendára a funkciou priameho kontaktovania organizátora (*Message to host*).
- **Identifikované nedostatky a technické bariéry:**
  - **Kritická nestabilita aplikácie:** Pri testovaní mobilnej aplikácie boli zaznamenané závažné chyby v procese autentifikácie (*Login failed*) a zlyhania pri verifikácii používateľa cez mobilné číslo, čo znemožňuje plnohodnotné využívanie služieb.
  - **Lokalizačné problémy:** Napriek deklarovanému globálnemu dosahu systém v niektorých prípadoch nedokáže korektne identifikovať menšie mestá, čo núti používateľa k prehliadaniu v režime hosťa (*Guest mode*).
  - **Absencia sociálnej interakcie:** Hoci platforma umožňuje nákup lístkov a sledovanie udalostí, postráda funkciu reálneho skupinového chatu medzi účastníkmi a transparentné zobrazenie zoznamu prihlásených osôb.
  - **Zameranie obsahu:** Databáza je orientovaná primárne na veľké komerčné a spoločenské akcie, čím opomína drobné, spontánne komunitné aktivity.

Zdroje:
`https://play.google.com/store/apps/details?id=com.amitech.allevents`
`https://allevents.in/`

2.4 Hapsy

Aplikácia *Hapsy* sa profiluje ako agregátor podujatí, ktorého cieľom je zjednotiť ponuku z rôznych platforiem (Facebook, Eventbrite, Meetup, GoOut) do jedného interaktívneho zobrazenia na mape. Jej hlavnou ambíciou je eliminovať potrebu prepínania medzi viacerými aplikáciami.

- **Kľúčové funkcie:** Agregácia dát z externých zdrojov, personalizované upozornenia na základe preferencií používateľa, ukladanie obľúbených udalostí a prehľadná mapa mesta.
- **Identifikované obmedzenia a používateľská kritika:**

- **Geografická limitácia:** Podľa spätnej väzby používateľov trpí aplikácia nedostatkom lokálneho obsahu mimo hlavných miest. Algoritmus sa zameriava na veľké komerčné databázy, čím opomína komunitný život v menších regiónoch.
- **Absencia tvorby obsahu:** Systém funguje primárne ako prehliadač (*viewer*). Bežný používateľ nemá možnosť vytvoriť vlastnú udalosť, čo znemožňuje organizáciu spontánnych aktivít (napr. spomínané amatérske športové zápasy).
- **Technická nedokonalosť:** Používateľské recenzie naznačujú prítomnosť kritických chýb v rozhraní (*UI bugs*), ako sú problémy s posúvaním obsahu (*scrolling*), čo znižuje komfort používania.
- **Obmedzená interakcia:** Podobne ako pri iných agregátoroch, aj tu chýba sociálna vrstva – možnosť priamej komunikácie medzi účastníkmi alebo správa tímov.

Zdroje:
`https://play.google.com/store/apps/details?id=com.hapsy.app`
`https://mapibara.com/`

2.5 Mapibara

Platforma *Mapibara* predstavuje minimalistický prístup k objavovaniu podujatí, pričom kladie absolútny dôraz na mapové rozhranie a anonymitu používateľa. Na rozdiel od komplexných sociálnych sietí sa zbavuje povinných registrácií a zameriava sa na okamžitý prístup k informáciám.

- **Silné stránky dizajnu:** Extrémne jednoduché a prehľadné používateľské rozhranie (*UI*), kde mapa zaberá celú plochu obrazovky. Absencia prihlasovacieho procesu odstraňuje vstupnú bariéru pre nových používateľov.
- **Identifikované kritické nedostatky:**
  - **Absencia obsahu:** Pri praktickom testovaní platforma vykazuje takmer nulovú naplnenosť podujatiami, čo ju robí v súčasnom stave nepoužiteľnou.
  - **Chýbajúca interakcia:** Kvôli absencii používateľských profilov nie je možné s účastníkmi komunikovať, spravovať kapacity ani riešiť logistiku (napr. spomínané rozdelenie do tímov pri športe).
  - **Pasivita systému:** Platforma slúži len ako statický prehliadač bodov na mape bez možnosti aktívneho zapojenia komunity.

Ponaučenie pre vlastný návrh: Analýza Mapibary ukazuje, že samotná vizualizácia na mape nestačí. Pre úspech aplikácie je nevyhnutné skombinovať **jednoduchosť mapy** so **sociálnou interakciou** (prihlásenie, chat), aby používatelia mali dôvod podujatia nielen hľadať, ale aj vytvárať a navštevovať.

2.6 Zhrnutie analýzy súčasného stavu

Na základe podrobnej analýzy dominantných platforiem (Facebook Events, Meetup, AllEvents, Hapsy a Mapibara) je možné definovať súbor štandardných funkcií, ktoré tvoria základ moderných systémov na správu podujatí, ale zároveň identifikovať kritické funkčné medzery.

Štandardné funkcie prítomné v analyzovaných riešeniach: Väčšina spomenutých aplikácií disponuje nasledujúcim funkčným základom:

- **Objavovanie a vizualizácia:** Interaktívna mapa s podujatiami, vyhľadávanie podľa kategórií a pokročilé filtrovanie.
- **Informačná vrstva:** Detailný popis podujatia, fotogaléria, profil organizátora a zdieľanie (*sharing*) na iné platformy.
- **Personalizácia:** História navštívených udalostí, systém odporúčaných podujatí (*Recommended for you*) a push notifikácie.
- **Interakcia:** Možnosť vytvorenia vlastnej verejnej alebo súkromnej udalosti po predchádzajúcej registrácii.

Identifikácia funkčných medzier (Inovačný potenciál): Napriek širokej škále funkcií analýza odhalila oblasti, ktoré súčasné riešenia pokrývajú nedostatočne, čo vytvára priestor pre vývoj nového, špecializovaného riešenia:

- **Dynamický manažment kapacity:** Absencia automatizovaného systému náhradníkov (*Waiting List*). Väčšina platforiem neponúka riešenie pre prípady, kedy je kapacita naplnená a nie je možné automaticky osloviť ďalšieho záujemcu v poradí pri uvoľnení miesta.
- **Verifikácia reálnej účasti:** Kritický nedostatok nástrojov na potláčanie *no-show* efektu (používatelia, ktorí potvrdia účasť, ale neprídu). Chýbajú mechanizmy ako **GPS Check-in** alebo verifikácia prítomnosti na mieste činu.
- **Komunitná logistika:** Absentujú nástroje na správu spoločného vybavenia (*Inventory sharing* – napr. kto prinesie športové náčinie) a funkcia reálneho skupinového chatu pre všetkých účastníkov v reálnom čase.

- **Moderovanie a bezpečnosť:** Pri raste používateľmi generovaného obsahu chýbajú v mnohých aplikáciách citlivé nástroje na nahlasovanie (*reporting*) a včasné blokovanie nevhodného obsahu alebo používateľov.

- **Regionálna relevantnosť:** Nízka hustota obsahu v menších mestách a regiónoch mimo hlavných metropol, čo je dôsledkom orientácie veľkých platforiem na komerčné a masové podujatia.

2.7 Syntéza poznatkov a východiská pre vlastný návrh

Na základe vykonanej analýzy existujúcich riešení je možné konštatovať, že hoci trh ponúka robustné platformy, žiadna z nich nepokrýva komplexne potreby malých, spontánnych komunít s dôrazom na logistiku a potláčanie negativity (napr. spomínaný *no-show* efekt).

Pre lepšiu prehľadnosť sú kľúčové rozdiely medzi analyzovanými platformami a navrhovaným riešením zhrnuté v Tabuľke **??**.

Tabuľka 3: Porovnanie kľúčových funkcií existujúcich platforiem a navrhovanej aplikácie

Funkcia / Platforma	**FB**	**Meetup**	**AllEvents**	**Hapsy**	**Mapibara**
Mapa ako primárne UI	Nie	Čiastočne	Áno	Áno	Áno
Tvorba udalostí	Áno	Platené	Áno	Nie	Nie
Skupinový chat	Nie	Áno	Nie	Nie	Nie
Manažment náhradníkov	Nie	Áno	Nie	Nie	Nie
Ochrana „No-show“	Nie	Nie	Nie	Nie	Nie
Zameranie na komunity	Áno	Áno	Nie	Nie	Nie

Definovanie smerovania vlastného riešenia Zistené nedostatky konkurenčných riešení priamo formujú funkčné požiadavky na moju aplikáciu. Vlastné riešenie sa bude odlišovať predovšetkým v troch pilieroch:

1. **Interaktivita namiesto pasivity:** Integrácia komunikačného modulu (chat) priamo do detailu udalosti, čo umožní operatívne riešiť zmeny (napr. počasie, meškanie) v reálnom čase.

2. **Zvýšenie kredibility účastníkov:** Implementácia overovacích mechanizmov (napr. potvrdzovanie prítomnosti na mieste cez polohu zariadenia), čo motivuje používateľov k reálnej účasti.

3. **Minimalizmus a dostupnosť:** Odstránenie finančných bariér pre organizátorov a zachovanie čistého mapového rozhrania pre okamžitý prehľad o dianí v okolí bez zbytočného informačného balastu.

Tento prístup umožní vytvoriť nástroj, ktorý nie je len „nástenkou podujatí“, ale živým ekosystémom pre lokálne komunity.
Zhrnutím možno konštatovať, že existuje dopyt po riešení, ktoré by premostilo priepasť medzi statickou informáciou o udalosti a aktívnou logistickou podporou komunitných aktivít.

3 Použité technológie

3.1 Vývojové prostredie a platforma

3.1.1 Operačný systém Android

Táto práca je zameraná na vývoj mobilnej aplikácie pre operačný systém **Android**. Dôvodom zvolenia tejto platformy je jej dominantné postavenie na trhu mobilných zariadení, kde podľa aktuálnych štatistík zaujíma viac ako 70 % podiel. Android ponúka široké možnosti prispôsobenia a hlbokej integrácie s hardvérovými službami (ako sú lokalizačné služby), čo je kľúčové pre praktickú časť tejto práce.
(Zdroj: `https://gs.statcounter.com/android-version-market-share`)

3.1.2 Distribúcia a ekosystém platforiem

Voľba operačného systému ovplyvňuje nielen technologický zásobník, ale aj stratégiu uvedenia na trh. Platforma Android je často preferovaná pre nasadenie minimálneho životaschopného produktu (*MVP*) kvôli flexibilnejším pravidlám v obchode Google Play v porovnaní s uzavretým ekosystémom iOS.

Hoci existujú multiplatformové riešenia, pre účely tejto práce bol zvolený natívny prístup z dôvodu maximálnej optimalizácie výkonu a prístupu k systémovým API. Porovnanie prístupov na základe analýzy [5] uvádza Tabuľka **??**.

Tabuľka 4: Súhrnné porovnanie natívneho a multiplatformového vývoja

Metrika	**Native (Kotlin)**	**Flutter**	**React Native**	**KMM**
Veľkosť aplikácie	**Najmenšia**	Väčšia	Väčšia	Stredná
Rýchlosť štartu	**Najrýchlejšia**	Stredná	Pomalšia	Rýchla
Animácie	**Plynulé**	Plynulé	Riziko lagov	–
Zdieľanie kódu	Žiadne	95 % +	90 % +	Logika
Spotreba batérie	**Nízka**	Stredná	Vysoká	Nízka

3.2 Programovací jazyk Kotlin

Kotlin je moderný, staticky typovaný jazyk, ktorý je od roku 2019 preferovanou voľbou spoločnosti Google pre vývoj aplikácií na platformu Android. Medzi jeho kľúčové prínosy pre túto prácu patria:

- **Null Safety:** Výrazne znižuje riziko pádov aplikácie (chyba *NullPointerException*) vďaka striktnému rozlišovaniu nulovateľných typov už v čase kompilácie.

- **Coroutines:** Umožňujú efektívne vykonávanie asynchrónnych úloh (napr. sťahovanie dát o podujatiach z cloudu) bez blokovania hlavného vlákna a zamŕzania používateľského rozhrania.
- **Stručnosť:** Redukcia nadbytočného kódu (*boilerplate*) umožňuje rýchlejšiu implementáciu biznis logiky odporúčacieho algoritmu.

3.3 UI Toolkit: Jetpack Compose

Na tvorbu používateľského rozhrania bol zvolený moderný deklaratívny toolkit **Jetpack Compose**. Na rozdiel od staršieho systému založeného na XML súboroch, Compose umožňuje definovať UI priamo v jazyku Kotlin.

- **Deklaratívny prístup:** Programátor opisuje výsledný stav rozhrania, pričom framework sa automaticky stará o jeho prekreslenie pri zmene dát (napr. pri aktualizácii zoznamu podujatí).
- **Komponentový dizajn:** UI je vyskladané z malých, znovupoužiteľných funkcií (*Composables*), čo uľahčuje údržbu a testovanie aplikácie.

3.4 Vývojové nástroje

Android Studio je oficiálne vývojové prostredie (IDE) bohaté na funkcie pre vývoj aplikácií pre Android, ktoré integruje nástroje na návrh, kódovanie, tvorbu a testovanie. Ponúka natívnu podporu pre Kotlin, Java a C++, rozsiahlu rozšíriteľnosť pluginov a bezproblémovú emuláciu naprieč typmi zariadení. Pravidelné aktualizácie zabezpečujú kompatibilitu s novými verziami systému Android, pokročilé ladenie a analýzu výkonu v jednotnom prostredí a preto je ideálnou voľbou pre tento projekt.

3.5 Technická špecifikácia a použité knižnice

V tejto časti si spravil kus dobrej roboty, ale ak ideme podľa tvojho plánu (najprv vymazať zbytočné, potom skracovať), máme tu jasných kandidátov na "vyhadzov".

1. Čo je v tejto časti úplne zbytočné (MAZAŤ): Definícia Android SDK: Vety o tom, že SDK obsahuje knižnice a emulátory a slúži na transformáciu kódu do balíka, sú veta pre prvákov. V inžinierskej práci o mobilnom vývoji sa predpokladá, že čitateľ vie, čo je SDK.

Verdikt: Vymaž celú prvú podkapitolu

3.5.1 Android SDK a správa verzií

. Nechaj len tie odrážky s verziami.

Marketingové reči o Material Design 3: Vety typu „...podporuje osvedčené postupy“, „...zefektívnenie spolupráce medzi dizajnom a vývojom“ alebo „...rýchle budovanie estetických produktov“ sú čistý marketingový text z webu Googlu.

Verdikt: Tieto vety letia von. Sú to prázdne frázy, ktoré oponent vníma ako vatu.

Definícia "targetSdk": Vysvetľovanie rozdielu medzi minSdk a targetSdk (posledný odsek pod odrážkami) je v diplomovke nepotrebné.

Verdikt: Vymaž tento vysvetľujúci odsek.

2. Ako to bude vyzerať po vymazaní (len čisté fakty): Tento text má teraz polovicu dĺžky, ale dvojnásobnú informačnú hodnotu:

Útržok kódu

3.6 Technická špecifikácia a dizajn

3.6.1 Konfigurácia Android SDK

V rámci projektu sú definované úrovne rozhraní API, ktoré určujú rozsah kompatibility a dostupné systémové funkcie:

- **minSdk 26 (Android 8.0):** Zvolené ako optimálny kompromis pre podporu moderných API (notifikačné kanály, JobScheduler) pri zachovaní pokrytia nad 90 % zariadení.
- **targetSdk 35 (Android 15):** Zabezpečuje súlad s najnovšími bezpečnostnými štandardmi a protokolmi ochrany súkromia vyžadovanými pre Google Play.

3.6.2 Vizuálne rozhranie Material Design 3

Používateľské rozhranie je postavené na štandarde **Material Design 3** s využitím knižnice `androidx.compose.material3`. Hlavné prínosy integrácie zahŕňajú:

- **Komponenty:** Využitie systémových prvkov ako `ModalNavigationDrawer` pre navigáciu a `ElevatedCard` pre zobrazenie detailov podujatí.
- **Adaptivita:** Natívna podpora pre rôzne hustoty pixelov a orientácie displeja.
- **Dynamické témy:** Podpora pre *Material You*, umožňujúca prispôsobenie farebnej palety aplikácie nastaveniam systému.

3.7 Externé softvérové knižnice

Pre rozšírenie základných funkcií systému Android a zabezpečenie plynulého používateľského zážitku boli do projektu integrované špecializované knižnice. Tieto nástroje riešia kritické úlohy od navigácie v deklaratívnom prostredí až po nízkoúrovňovú komunikáciu s hardvérom kamery.

3.7.1 Navigation Compose (2.8.5)

Knižnica **Navigation Compose** poskytuje infraštruktúru pre riadenie toku obrazoviek v aplikáciách postavených na Jetpack Compose. Umožňuje definovať navigačný graf priamo v kóde Kotlin, čím nahrádza staršie riešenia založené na XML súboroch a fragmentoch.

- **NavHost:** Slúži ako vizuálny kontajner, ktorý dynamicky vymieňa jednotlivé kompozitné funkcie (*composables*) na základe aktuálnej trasy.
- **NavController:** Centrálny objekt, ktorý spravuje zásobník obrazoviek (*backstack*) a zabezpečuje správnu reakciu na systémové tlačidlo „Späť“.
- **Typová bezpečnosť:** Verzia 2.8.5 prináša natívnu podporu pre typovo bezpečnú navigáciu, čo eliminuje chyby pri prenose argumentov (napr. ID podujatia) medzi obrazovkami.

3.7.2 CameraX (1.3.4)

CameraX je moderná knižnica zameraná na prácu s fotoaparátom, ktorá odstraňuje zložitosť spojenú so správou špecifík hardvéru od rôznych výrobcov. Je postavená na koncepte prípadov použitia (*use-cases*), z ktorých boli v projekte implementované najmä:

- **Preview:** Zabezpečuje plynulý živý náhľad obrazu z kamery priamo v používateľskom rozhraní.
- **Image Analysis:** Poskytuje prístup k obrazovým dátam v reálnom čase, čo je kľúčové pre **algoritmy na skenovanie a dekódovanie QR kódov** pri vstupe na podujatia.
- **Image Capture:** Umožňuje zachytávanie fotografií (napr. pre profilové fotky alebo dokumentáciu podujatia) s automatickou optimalizáciou expozície a zaostrenia.

3.7.3 Coil (2.7.0)

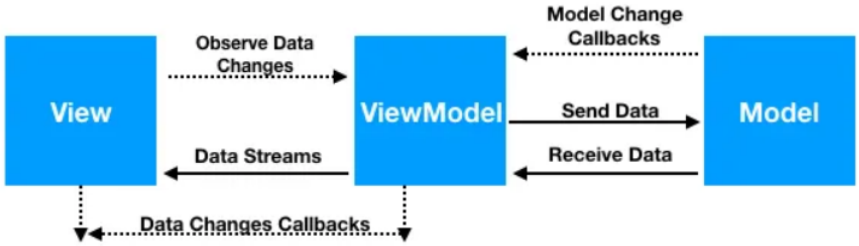
Názov **Coil** predstavuje akronym pre *Coroutine Image Loader*. Ide o efektívnu knižnicu na asynchrónne načítavanie obrázkov z cloudu (Firebase Storage), ktorá je plne optimalizovaná pre prostredie Kotlin Coroutines.

- **Optimalizácia výkonu:** Knižnica automaticky vykonáva operácie ako cachovanie na disk a do pamäte RAM, zmenu mierky obrázkov (*downsampling*) a inteligentné rušenie sieťových požiadaviek pri odscrollovaní prvku z obrazovky.
- **Integrácia s Compose:** Vďaka natívnej podpore pre Jetpack Compose umožňuje zobrazenie obrázkov s minimálnym množstvom riadiaceho kódu (*boilerplate*), čím zvyšuje čitateľnosť projektu.

3.8 Architektúra projektu

V projekte využívame architektúru **MVVM** (*Model-View-ViewModel*) v kombinácii s **Repository pattern**. Model MVVM pozostáva z troch základných komponentov, z ktorých každý má jasne definovanú zodpovednosť:

- **Model:** Reprezentuje dátovú vrstvu a biznis logiku. V našom prípade sem patrí komunikácia s Firebase a správa lokálnych dát.
- **View (Zobrazenie):** Používateľské rozhranie, ktoré pasívne zobrazuje dáta a odosiela akcie používateľa do ViewModelu.
- **ViewModel (Model zobrazenia):** Pôsobí ako prostredník, ktorý pripravuje dáta z modelu pre zobrazenie a udržiava stav aplikácie nezávisle od životného cyklu rozhrania.



Obr. 1: Diagram architektúry MVVM s Repository pattern v systéme Android

Hlavným dôvodom voľby MVVM sú jeho prínosy pre stabilitu projektu, najmä **zvýšená testovateľnosť** (možnosť testovať logiku bez UI), **jednoduchšia údržba** vďaka

oddeleniu zodpovedností a **odolnosť voči zmenám konfigurácie** (napr. otočenie displeja), kedy ViewModel chráni dáta pred stratou.

3.8.1 Moderný prístup: MVVM v kombinácii s Jetpack Compose

V súlade s princípmi čistého návrhu (*Clean Architecture*) opísanými v [**sks_medium_2024**], je v projekte implementovaný vzor MVVM s dôrazom na **jednosmerný tok dát** (*Unidirectional Data Flow - UDF*). Tradičný mechanizmus *Data Bindingu* známy zo starších verzií Androidu je tu nahradený moderným reaktívnym prístupom:

- **State Hoisting (Vynášanie stavu):** ViewModel uchováva kompletný stav obrazovky v podobe imutabilného objektu (napr. *EventsUiState*). UI komponenty sú takmer bezstavové (*stateless*), čo uľahčuje ich znovupoužiteľnosť.
- **Reaktívne pozorovanie (StateFlow):** Používateľské rozhranie je prepojené s *StateFlow* vo ViewModele. Akákoľvek zmena v dátach (napr. aktualizácia zoznamu podujatí) vyvolá okamžitú **rekompozíciu** (automatické prekreslenie) dotknutých častí rozhrania.
- **Spracovanie udalostí (Events):** Všetky akcie používateľa sú prenášané ako udalosti smerom k ViewModelu. View nikdy neupravuje dáta priamo, čo zaručuje, že *ViewModel* zostáva jediným zdrojom pravdy (*Single Source of Truth*).

3.9 Platforma Firebase

Firebase predstavuje komplexnú platformu cloudových služieb od spoločnosti Google, ktorá je navrhnutá na zrýchlenie vývoja, zvýšenie bezpečnosti a zabezpečenie škálovateľnosti moderných aplikácií.

3.9.1 Dôvody voľby a kľúčové výhody

Voľba tejto platformy pre účely mojej práce vychádza z jej schopnosti riešiť zložité vývojové výzvy pomocou integrovaných produktov:

- **Efektívna autentifikácia:** Poskytuje robustné knižnice pre správu používateľov, overovanie hesiel a integráciu poskytovateľov tretích strán (Google, GitHub atď.).
- **Škálovateľné úložisko:** Cloud Storage umožňuje efektívnu prácu s používateľským obsahom, ako sú obrázky a videá, s vysokou úrovňou zabezpečenia.
- **Real-time synchronizácia:** NoSQL databázy (Firestore, Realtime Database) umožňujú synchronizáciu dát naprieč klientmi v reálnom čase, a to aj v offline režime.

- **Bezpečnosť a správa:** Flexibilné bezpečnostné pravidlá chránia dáta pred neoprávneným prístupom priamo na úrovni cloudu.
- **Serverless logika:** Cloud Functions umožňujú rozšíriť funkcionalitu aplikácie o asynchrónne spracovanie na pozadí bez nutnosti správy vlastného servera.

3.10 Firebase Authentication

Firebase Authentication poskytuje backendové služby na autentifikáciu používateľov v aplikácii. Podporuje overovanie pomocou hesiel, telefónnych čísiel a populárnych poskytovateľov identít (Google, Facebook, Twitter).

3.10.1 Mechanizmus fungovania a správa tokenov

Proces autentifikácie vo Firebase Authentication je založený na výmene a overovaní tokenov, čo zabezpečuje bezstavovú komunikáciu medzi klientom a backendom. Celý proces prebieha v niekoľkých krokoch:

- **Prihlásenie a vytvorenie ID tokenu:** Po úspešnom overení identity (napr. zadaním hesla) Firebase servery vygenerujú *Firebase ID Token*. Ide o *JSON Web Token* (JWT), ktorý obsahuje informácie o identite používateľa (UID, e-mail, čas prihlásenia). Tento token je krátkodobý a jeho platnosť je štandardne 1 hodina.
- **Refresh Token:** Súčasne s ID tokenom klient dostane aj *Refresh Token*. Na rozdiel od ID tokenu, refresh token má dlhú platnosť. Firebase SDK ho automaticky ukladá do lokálneho úložiska prehliadača alebo zariadenia a používa ho na automatické získanie nového ID tokenu po vypršaní toho starého, čím zabezpečuje, že používateľ nemusí opakovane zadávať prihlasovacie údaje.
- **Autorizácia požiadaviek:** Pri každej požiadavke na chránené zdroje (napr. Cloud Firestore alebo vlastný backend) klientska aplikácia pripojí aktuálny ID token do hlavičky požiadavky. Prijímajúca strana následne overí integritu a platnosť tohto tokenu pomocou verejných kľúčov Google.
- **Bezpečnostné pravidlá:** Identita extrahovaná z tokenu je priamo dostupná v bezpečnostných pravidlách Firebase (*Security Rules*), čo umožňuje jednoducho definovať prístupové práva na úrovni jednotlivých používateľov (napr. `request.auth.uid == userId`).

https://firebase.google.com/docs/auth

3.10.2 Prepojenie s Cloud Firestore a dokument users/{uid}

Hoci Firebase Authentication spravuje prihlasovacie údaje, neukladá dodatočné informácie o používateľovi (napr. adresu, telefónne číslo, preferencie alebo rolu v systéme). Na tento účel sa v kombinácii s autentifikáciou využíva databáza *Cloud Firestore*.

- **Identifikátor UID:** Každý používateľ po registrácii získa unikátny identifikátor (*Unique Identifier - UID*). Tento kľúč je nemenný a slúži ako most medzi autentifikačnou službou a databázou.
- **Dokument users/{uid}:** Bežným architektonickým vzorom je vytvorenie kolekcie s názvom `users`, kde názov každého dokumentu zodpovedá konkrétnemu `uid` používateľa. Týmto spôsobom je zabezpečené, že každý prihlásený používateľ má priradený práve jeden profilový dokument.
- **Bezpečnostná integrácia:** Vďaka tomu, že sa názov dokumentu zhoduje s `uid`, je možné v *Firestore Security Rules* jednoducho definovať prístup:

  ```
  match /users/{userId} {
    allow read, write: if request.auth != null && request.auth.uid == userId;
  }
  ```

  Tento zápis zabezpečí, že k dokumentu v ceste `users/123` sa dostane výhradne používateľ, ktorého `uid` je `123`.

3.11 Firebase Firestore

Cloud Firestore je flexibilná, škálovateľná NoSQL databáza určená pre mobilný a webový vývoj. Udržiava dáta synchronizované medzi klientskymi aplikáciami prostredníctvom reaktívnych poslucháčov (*realtime listeners*).

Kľúčovou vlastnosťou pre túto prácu je **offline podpora**, ktorá umožňuje aplikácii plynule fungovať aj pri nestabilnom internetovom pripojení tým, že lokálne cachuje potrebné dáta a synchronizuje ich hneď po obnovení siete.

3.12 Firebase Storage

Cloud Storage pre Firebase je výkonná a nákladovo efektívna služba určená na ukladanie objektov, ako sú fotografie podujatí či profilové obrázky používateľov. Súpravy SDK zabezpečujú vysokú mieru spoľahlivosti pri nahrávaní a sťahovaní súborov bez ohľadu na kvalitu sieťového pripojenia, pričom prístup k súborom je striktne riadený integračnými pravidlami Firebase Authentication.

3.13 Technológia QR kódov

Existuje nekonenečne veľa spôsobov ako overiť vstup od podujatie, jedno z najefektívnejších je využitie QR kódov. QR kód (*Quick Response Code*) je dvojrozmerný čiarový kód, kotrý nachádza uplatnenie vo všetkých oblastí digitálnej komunikácie a môže byť použitý na overenie vstupu na podujatie.

3.13.1 Kľúčové vlastnosti a štruktúra

QR kód sa od klasických lineárnych čiarových kódov líši niekoľkými zásadnými charakteristikami:

- **Vysoká hustota dát:** Dokáže uchovať oveľa väčšie množstvo informácií na malej ploche (text, URL adresy, kontaktné údaje).
- **Podpora znakov:** Okrem číslic a latinky podporuje aj zložitejšie znakové sady (napr. binárne dáta alebo Kanji).
- **Odolnosť voči poškodeniu:** Vďaka vstavanej korekcii chýb (Reed-Solomonov kód) je možné QR kód správne prečítať, aj keď je časť jeho plochy znečistená alebo poškodená.
- **Otvorený štandard:** Patent na túto technológiu bol uvoľnený pre verejnosť, čo znamená, že QR kódy môže generovať a používať ktokoľvek bezplatne.

3.13.2 Využitie v mobilných zariadeniach

V súčasnosti je väčšina moderných smartfónov vybavená kamerami s natívnou podporou čítania QR kódov. Tento mechanizmus sa najčastejšie využíva na:

- **Rýchly prístup k informáciám:**
- **Bezkontaktné interakcie:**
- **Autentifikáciu:**

(Zdroj: `https://www.iso.org/obp/ui/en/#iso:std:iso-iec:18004:ed-4:v1:en`)

ZXing Core (3.5.3) *ZXing* je široko používaná open-source knižnica na prácu s čiarovými kódmi [**zxing_repo**]. V rámci aplikácie sa využíva jej jadro (*Core*) na **generovanie QR kódov**.

- **Proces:** Knižnica transformuje unikátny identifikačný reťazec (token) z databázy do matice bitov (*BitMatrix*), ktorá je následne vykreslená ako bitmapový obrázok na displeji zariadenia.

- **Konfigurácia:** Umožňuje presné nastavenie kódovania (UTF-8) a úrovne korekcie chýb, čo zabezpečuje čitateľnosť kódu aj pri nižšom jase displeja.

https://github.com/zxing/zxing

ML Kit Barcode Scanning (17.3.0) Pre reverzný proces – dekódovanie QR kódu – bola zvolená sada *ML Kit* [**mlkit_barcode**]. Na rozdiel od starších knižníc využíva modely **strojového učenia** na detekciu kódov v reálnom čase.

- **Výkon:** Algoritmus dokáže identifikovať a dekódovať QR kód takmer okamžite, a to aj v prípade, že je kód naklonený, rozmazaný alebo sa nachádza v pohybe.

- **Integrácia:** Knižnica úzko spolupracuje s rozhraním *CameraX*, kde spracováva prúd obrazových dát (*Image Analysis*) a v prípade nálezu kódu vráti jeho textovú hodnotu pre ďalšiu validáciu voči systému Firebase.

https://developers.google.com/ml-kit/vision/barcode-scanning

3.14 Cloudová infraštruktúra a Google Maps Platform

Aplikácia využíva služby **Google Cloud Platform (GCP)**, čo je komplexný súbor cloudových služieb. GCP umožňuje vývojárom a podnikom prevádzkovať aplikácie na bezpečnej, vysoko výkonnej a škálovateľnej infraštruktúre, https://www.geeksforgeeks.org/devops/google--cloud-platform-gcp/

Maps SDK for Android *Maps SDK for Android* umožňuje pridávať mapové podklady do aplikácie ako zobrazenia máp a reakcií na gestá. SDK poskytuje nástroje na vkladanie doplňujúcich informácií o polohách a podporuje interakciu používateľa prostredníctvom značiek (*Markers*), polygónov a iných prekrytí (*Overlays*).

Places SDK for Android *Places SDK for Android* umožňuje vytvárať aplikácie, ktoré kontextuálne reagujú na okolité podniky a iné miesta v blízkosti zariadenia. Tento nástroj dopĺňa základné geografické služby systému Android o bohatú databázu miest, ktoré majú pre používateľa konkrétny význam.

Kľúčové funkcie a rozhrania zahŕňajú:

- **Programový prístup k databáze:** SDK poskytuje prístup k informáciám o miestnych podnikoch, bodoch záujmu a aktuálnej polohe zariadenia.

- **Autocomplete:** Ponúka predpripravené komponenty (*Widgets*) na predpovedanie názvov miest v reakcii na vyhľadávacie dopyty používateľa.

- **Definícia miesta (*Place*):** Miesto je v rámci API reprezentované rozhraním *Place*. Objekt *Place* obsahuje údaje ako názov, adresa, ID miesta, geografická poloha, telefónne číslo, typ miesta či webová adresa.

Podobne ako Maps SDK, aj tento nástroj plne podporuje jazyky Kotlin a Java vrátane rozšírení pre pokročilý vývoj.

Google Sign-In Tento komponent spravuje autentifikačný tok *OAuth 2.0* a životný cyklus prístupových tokenov. Integruje prihlasovanie pomocou Google účtu, čím zjednodušuje overovanie identity používateľa bez nutnosti manuálnej správy prihlasovacích údajov.

Google Location Services Knižnica `play-services-location` je súčasťou služieb Google Play a poskytuje vysoko presné a efektívne sledovanie polohy zariadenia.

Konfigurácia a bezpečnosť rozhraní Prepojenie aplikácie s cloudovou infraštruktúrou je realizované prostredníctvom konfiguračného súboru `google-services.json` a unikátnych API kľúčov. Bezpečnosť týchto rozhraní je zabezpečená na dvoch úrovniach:

- **Technické obmedzenia:** Prístup k API je v *Google Cloud Console* striktne obmedzený na balík aplikácie (*Package Name*) a unikátny **SHA-1 odtlačok** podpisového certifikátu vývojára.
- **Separácia citlivých údajov:** API kľúče sú uložené mimo zdrojového kódu v súbore `local.properties`, ktorý je ignorovaný systémom správy verzií, a do projektu sú načítavané pomocou nástroja *Secrets Gradle Plugin*.

3.15 Cloudinary – správa digitálnych aktív

Pre efektívnu správu, optimalizáciu a doručovanie vizuálneho obsahu (obrázky podujatí, logá, profilové fotografie) bola do projektu integrovaná platforma **Cloudinary**. Ide o moderný systém správy digitálnych aktív, ktorý využíva umelú inteligenciu a pokročilé algoritmy na spracovanie obrazu [**cloudinary_blog**].

3.15.1 Kľúčové výhody a funkcie

Cloudinary poskytuje nástroje, ktoré presahujú možnosti bežného cloudového úložiska:

- **Automatická optimalizácia:** Systém pomocou AI automaticky upravuje formát, kvalitu a rozlíšenie obrázkov v reálnom čase podľa zariadenia používateľa, čo výrazne znižuje spotrebu dát a zrýchľuje načítavanie aplikácie.

- **Dynamické transformácie:** Umožňuje vykonávať úpravy (orezanie na štvorec, zmena veľkosti, pridanie vodoznaku) priamo prostredníctvom úpravy URL adresy obrázka, bez nutnosti manuálneho generovania viacerých verzií súborov.
- **Škálovateľnosť:** Infraštruktúra je navrhnutá tak, aby bezchybne doručovala vizuálny obsah vo veľkom meradle (*at scale*) pri zachovaní vysokej rýchlosti odozvy.

3.15.2 Porovnanie s Firebase Storage a ekonomické hľadisko

Hoci ekosystém Firebase, využívaný v tejto práci, ponúka vlastnú službu *Firebase Storage*, pre účely správy obrázkov bola zvolená platforma Cloudinary z nasledujúcich dôvodov:

1. **Nákladová efektivita:** Firebase Storage uplatňuje striktné limity na prenos dát a ukladanie
2. **Absencia natívnej manipulácie:** Firebase Storage slúži primárne ako „pasívne“ úložisko súborov

3.16 Návrh algoritmu odporúčania

Pre personalizáciu obsahu bol navrhnutý a implementovaný **váhový hybridný model odporúčania** (*Weighted Hybrid System*). Tento systém dynamicky kombinuje implicitné dáta z histórie interakcií používateľa s explicitnými preferenciami zadávanými v reálnom čase pomocou váhových koeficientov. Na rozdiel od čisto obsahového filtrovania (*Content-based filtering*) tento prístup umožňuje algoritmu okamžite reagovať na aktuálny zámer používateľa a efektívne predchádzať vzniku filtračných bublín prioritizáciou fixných atribútov podujatí (kategória, poloha, čas) voči dynamicky sa meniacemu používateľskému profilu.“

4 Návrh riešenia

V tejto sekcii sa venujeme návrhu riešenia aplikácie, počnúc definíciou funkčných a nefunkčných požiadaviek, cez grafické návrhy a diagramy, až po samotnú implementáciu a testovanie.

4.1 Funkcionálne a nefunkcionálne požiadavky

Definovanie funkcionálnych a nefunkcionálnych požiadaviek je kľúčovým krokom návrhu, pretože nám pomáha jasne stanoviť správanie aplikácie, ktoré by splnilo jej cieľ.

4.1.1 Funkcionálne požiadavky

Funkcionálne požiadavky definujú správanie systému a služby, ktoré používateľ od systému očakáva. Medzi konkrétne požiadavky navrhovanej aplikácie patrí:

- registrácia, prihlásenie a verifikácia používateľa,
- správa profilu (úprava avatara, mena, popisu a kontrola unikátnosti prezývky),
- vytvorenie novej udalosti s detailmi (fotografie, popis, miesto, čas, typ, kapacita),
- správa súkromných udalostí chránených heslom,
- prehliadanie udalostí na interaktívnej mape s automatickým skrývaním neaktuálnych podujatí,
- nastavenie polomeru zobrazovania udalostí v okolí a filtrovanie podľa preferencií,
- potvrdenie účasti a možnosť odhlásenia sa z udalosti,
- systém náhradníkov (Waitlist) s automatickým povýšením po uvoľnení miesta,
- Check-in systém (manuálny a cez QR kód),
- komunikácia medzi účastníkmi podujatia a správa priateľov,
- ukladanie udalostí do zoznamu obľúbených a zobrazenie histórie (navštívené, odporúčané),
- systém hodnotenia a zdieľania (*sharing*) udalostí,
- pozývanie používateľov prostredníctvom e-mailu a správa pozvánok,
- nastavenie času notifikácií a správa upozornení,
- systém nahlasovania nevhodného obsahu a penalizácia používateľov za nedodržanie účasti,
- lokalizácia prostredia aplikácie (zmena jazyka a preklady),
- odporúčací systém pre relevantné podujatia.

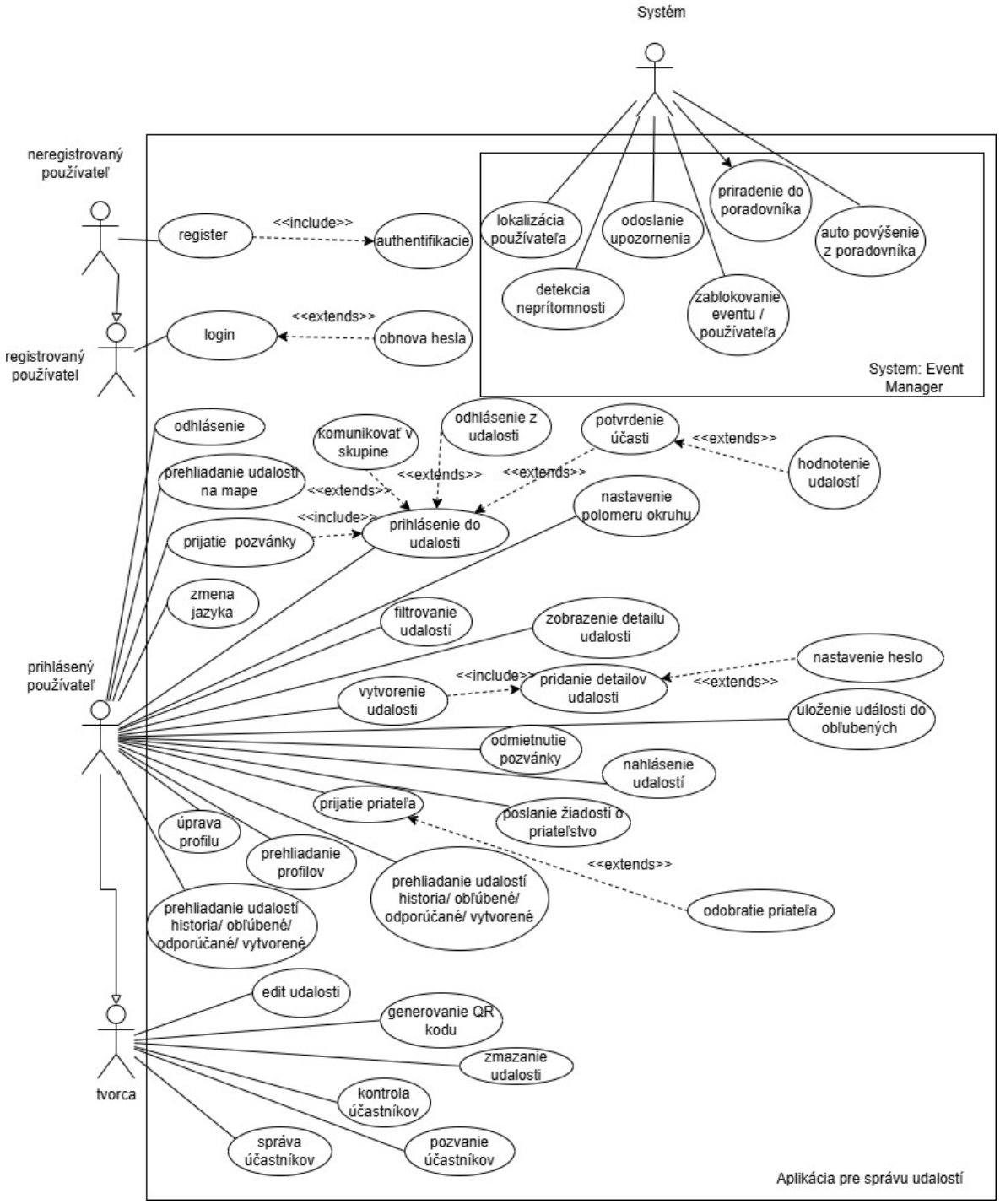
4.1.2 Nefunkcionálne požiadavky

Nefunkcionálne požiadavky definujú kvalitatívne vlastnosti a obmedzenia systému:

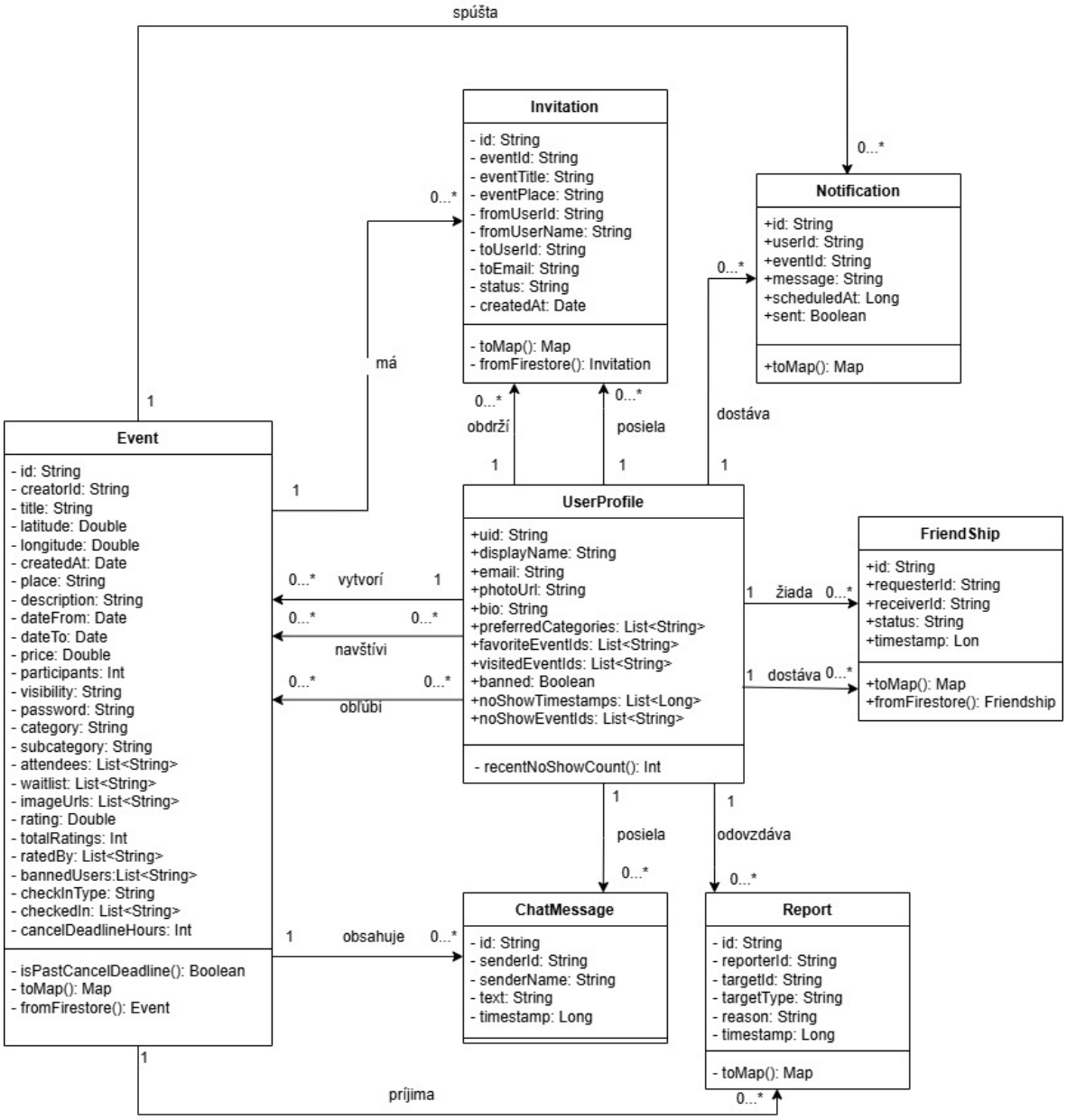
- jasná navigácia v používateľskom rozhraní bez potreby dodatočnej dokumentácie,
- korektné prispôsobenie rozhrania rôznym rozlíšeniam a pomerom strán Android zariadení,
- spracovanie dát a načítanie mapových podkladov v časovom limite do 2 sekúnd,
- garantovaná stabilita procesov na pozadí a spoľahlivosť systému prihlasovania,
- efektívne ukladanie dát s cieľom minimalizácie prenosov v rámci Firebase Firestore,
- energeticky šetrné spracovanie GPS súradníc používateľa,
- korektné vyžadovanie a správa systémových povolení (poloha, galéria, kalendár),
- zabezpečenie osobných údajov a šifrovaná komunikácia medzi klientom a serverom,
- možnosť synchrónneho používania aplikácie na viacerých zariadeniach pod jedným účtom,
- modulárna architektúra kódu umožňujúca jednoduchú rozšíriteľnosť systému,
- automatické zachovanie integrity dát po odstránení udalosti alebo zablokovaní používateľa.

4.2 UML diagramy

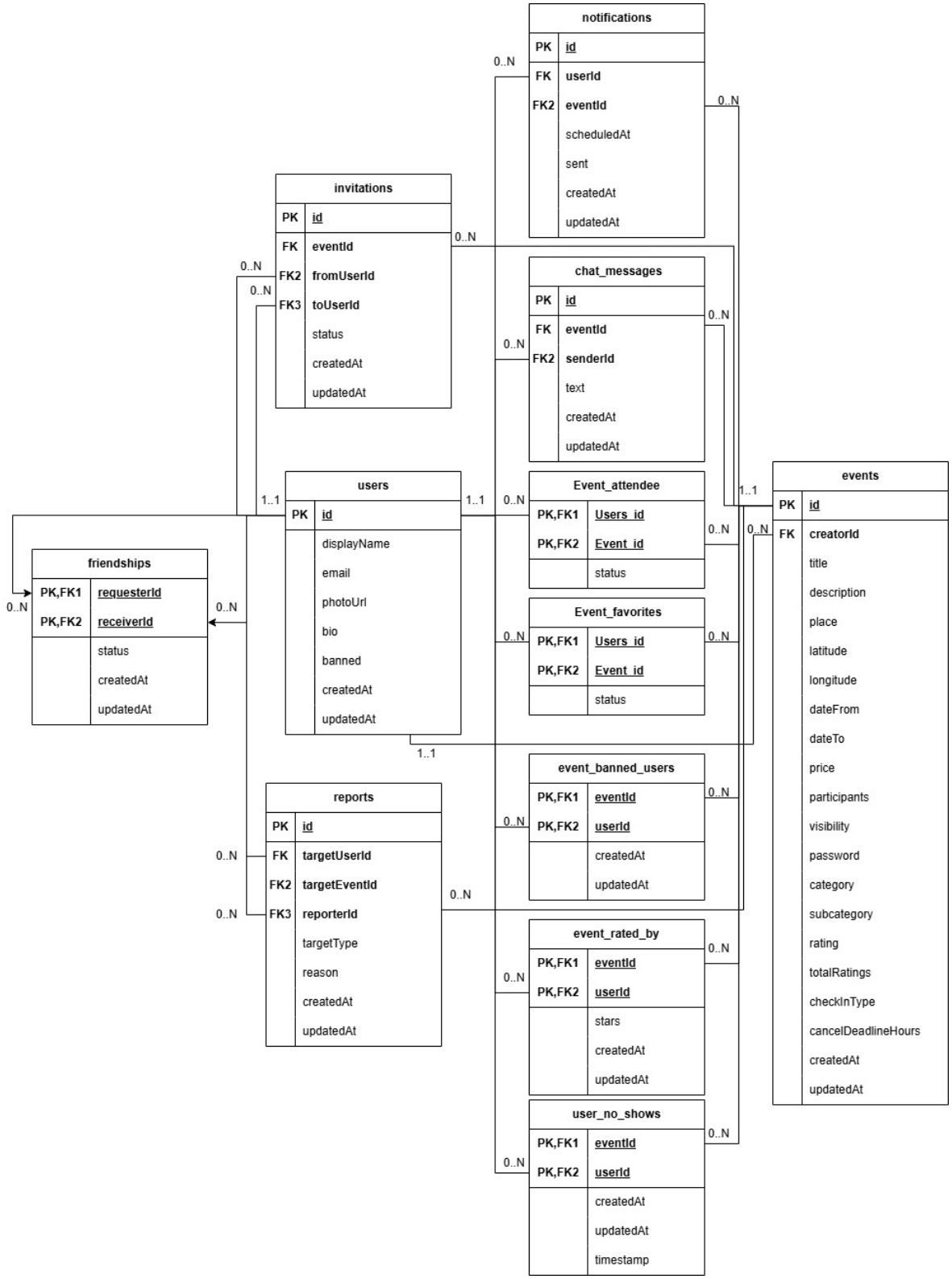
V tejto časti sú znázornené UML diagramy navrhovanej aplikácie, ktoré vizualizujú jej štruktúru a správanie.



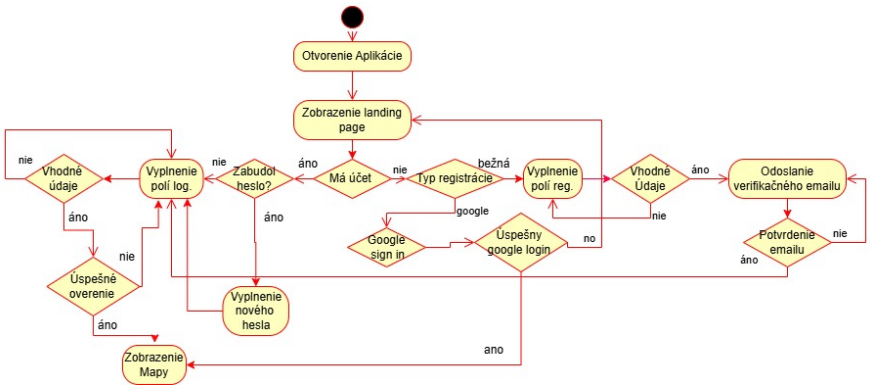
Obr. 2: Diagram prípadov použitia (Use Case diagram)



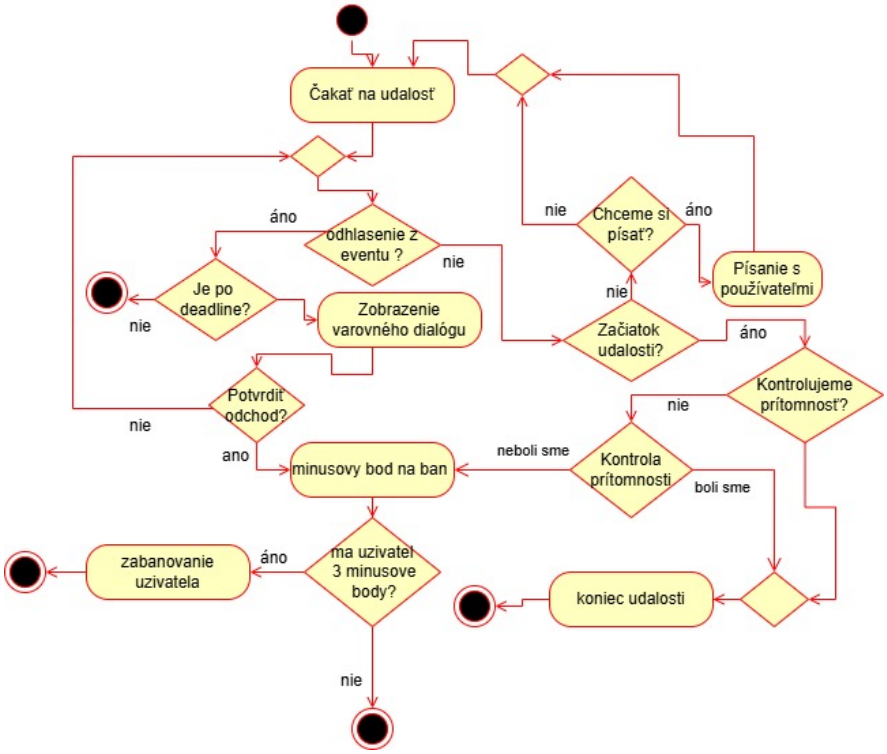
Obr. 3: Diagram tried (Class diagram)



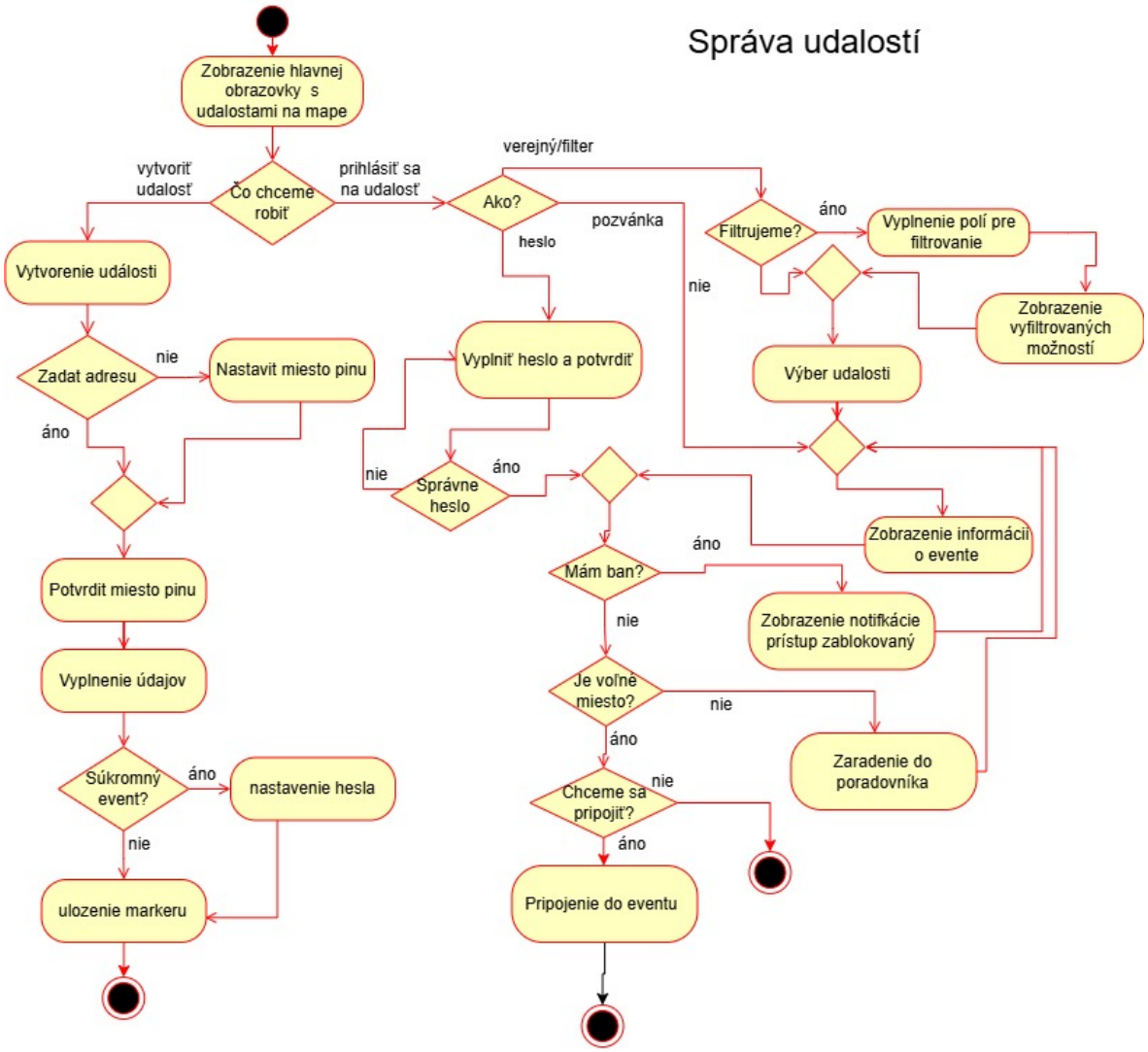
Obr. 4: Relačná databáza



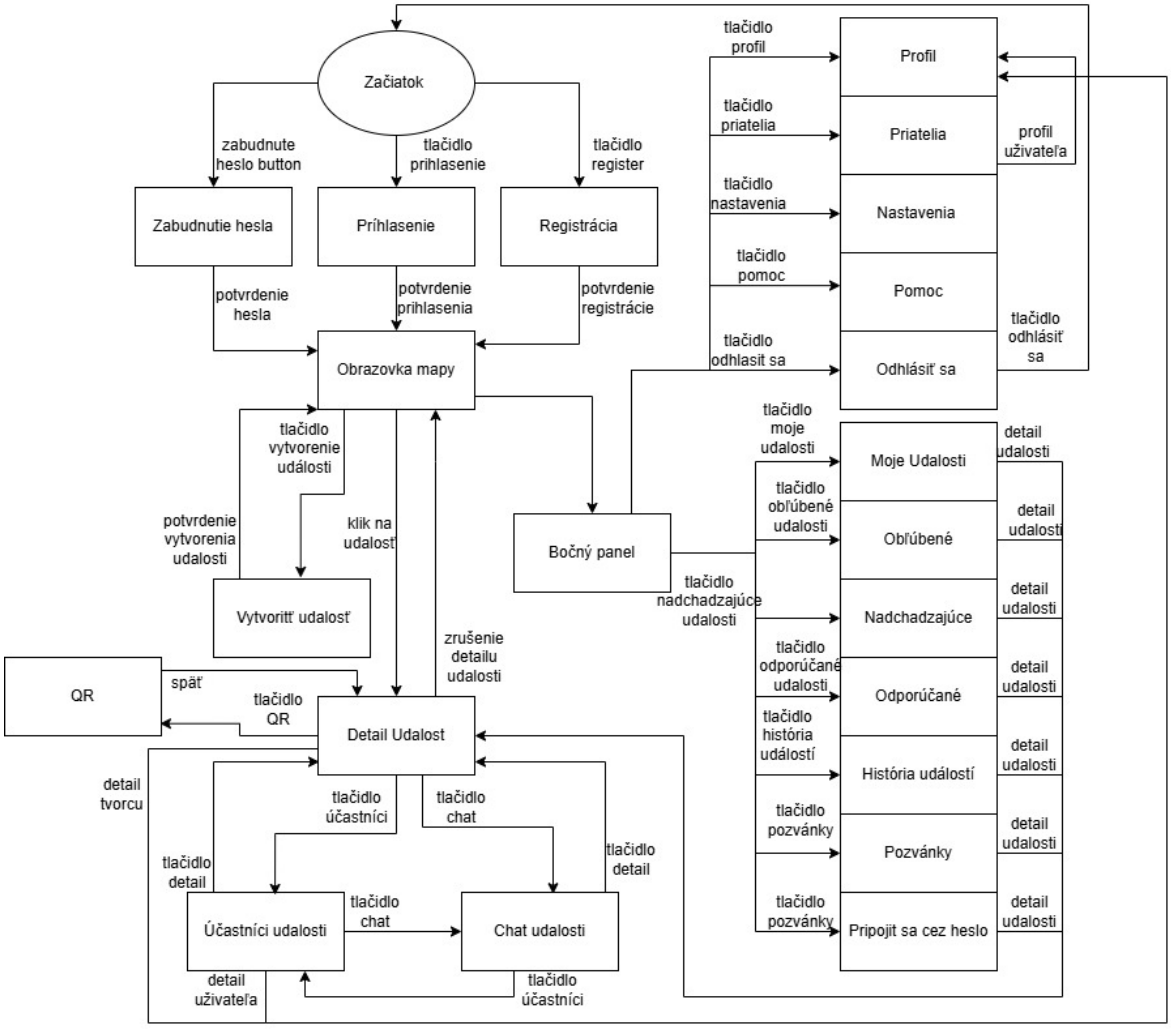
Obr. 5: Diagram aktivít – autentifikácia



Obr. 6: Diagram aktivít – účasť na udalosti



Obr. 7: Diagram aktivít – správa udalostí



Obr. 8: Navigačná lišta aplikácie

5 Implementacia

V tejto časti sa zameriame na samotnú implementáciu navrhovanej aplikácie, ktorá zahŕňa výber technológií, architektúru systému a detailný popis jednotlivých komponentov.

5.1 Autentifikácia a správa používateľov

Aplikácia je navrhnutá podľa architektúry **MVVM (Model-View-ViewModel)**, ktorá zabezpečuje jasné oddelenie biznis logiky od používateľského rozhrania. Implementácia autentifikácie prebieha na úrovni vrstvy *Model* (služba `AuthService`), zatiaľ čo validácia a spracovanie stavov sú riešené vo vrstve *ViewModel*.

5.1.1 Implementácia kľúčových funkcií

Služba `AuthService` zabezpečuje nízkoúrovňovú komunikáciu s Firebase. Využíva asynchrónne volania pomocou Kotlin korutín, čo umožňuje neblokujúce vykonávanie sieťových operácií. Registrácia nového používateľa zahŕňa nielen vytvorenie účtu, ale aj okamžitú aktualizáciu profilu o zobrazované meno (*displayName*):

```
suspend fun registerWithEmail(
    email: String,
    password: String,
    displayName: String
): FirebaseUser? {
    val credential = auth.createUserWithEmailAndPassword(email, password).await()
    credential.user?.updateProfile(
        userProfileChangeRequest { this.displayName = displayName }
    )?.await()
    return credential.user
}
```

Výpis kódu 1: Registrácia používateľa v AuthService

Prihlásenie cez Google účet prebieha prostredníctvom `GoogleSignInClient`, pričom po získaní `idToken` sa volá metóda `firebaseAuthWithGoogle`:

```
suspend fun firebaseAuthWithGoogle(idToken: String): FirebaseUser? {
    val credential = GoogleAuthProvider.getCredential(idToken, null)
    val result = auth.signInWithCredential(credential).await()
    return result.user
}
```

Výpis kódu 2: Prihlásenie cez Google v AuthService

Okrem externých poskytovateľov je implementované aj štandardné prihlásenie pomocou e-mailu a hesla:

```
suspend fun signInWithEmail(email: String, password: String) {
    auth.signInWithEmailAndPassword(email, password).await()
}
```

Výpis kódu 3: Prihlásenie pomocou e-mailu a hesla

5.1.2 Validácia vstupných údajov

Pred odoslaním požiadavky na server vykonáva `ViewModel` lokálnu validáciu na strane klienta. Týmto prístupom šetríme sieťovú prevádzku a zlepšujeme odozvu rozhrania. V rámci bezpečnosti sa kontroluje:

- formát e-mailovej adresy (validita syntaxe),
- komplexnosť hesla (minimálna dĺžka 6 znakov, prítomnosť veľkého písmena a špeciálneho znaku),
- zhoda hesla s jeho potvrdením pri registrácii.

```
passwordError = when {
    password.length < 6 -> "Heslo musí mať aspoň 6 znakov."
    !password.any { it.isUpperCase() } -> "Heslo musí obsahovať veľké písmeno."
    !password.any { !it.isLetterOrDigit() } -> "Heslo musí obsahovať špeciálny
    znak."
    else -> null
}
```

Výpis kódu 4: Validácia hesla v RegisterViewModel

5.1.3 Bezpečnostné obmedzenia a banovanie

Nad rámec štandardnej autentifikácie aplikácia implementuje kontrolu stavu používateľa. Pri každom prihlásení sa overuje atribút `banned` v dokumente používateľa v databáze Firestore. V prípade, že je používateľ zablokovaný, systém ho okamžite odhlási a zamedzí prístup k chránenému obsahu aplikácie.

5.2 Implementácia mapového modulu a geolokalizácie

Ako základ pre mapu je použitá knižnica *Google Maps SDK*, ktorá poskytuje všetky potrebné funkcionality pre prácu s geografickými dátami. Používateľ na hlavnej obrazovke vidí interaktívnu mapu, na ktorej sú jednotlivé udalosti zobrazené vo forme markerov.

5.2.1 Zobrazenie mapy a integrácia s Compose

Keďže `MapView` nie je natívny komponent prostredia *Jetpack Compose*, využíva sa `AndroidView` ako most (*bridge*) medzi klasickým *Android View* systémom a *Compose UI*. Mapa sa pri prvotnom načítaní predvolene centruje na súradnice Bratislavy.

```
val mapView = remember { MapView(context).also { it.onCreate(null) } }

AndroidView(factory = { mapView }, modifier = Modifier.fillMaxSize()) { mv ->
    mv.getMapAsync { map ->
        map.moveCamera(CameraUpdateFactory.newLatLngZoom(BRATISLAVA, 13f))
    }
}

```

Výpis kódu 5: Inicializácia MapView v prostredí Compose

`MapView` disponuje vlastným životným cyklom, ktorý je manuálne prepojený s životným cyklom obrazovky pomocou `DisposableEffect`, čím sa zabezpečí správna reakcia na udalosti *onPause*, *onResume* a *onDestroy*.

5.2.2 GPS poloha a správa oprávnení

Po udelení oprávnenia `ACCESS_FINE_LOCATION` aplikácia získa poslednú známu polohu zariadenia pomocou `FusedLocationProviderClient`. Kamera sa presunie na polohu používateľa iba raz — tento stav riadi príznak `mapMovedToUser` v `AppViewModel`, čím sa zabraňuje opakovanému presunu kamery pri každej rekomponzícii rozhrania.

5.2.3 Práca s markermi a interakcia

Markery sú synchronizované so stavom `appViewModel.activeEvents` prostredníctvom `snapshotFlow` — pri každej zmene zoznamu udalostí sa mapa reaktívne aktualizuje. Do vlastnosti `tag` každého markera sa ukladá `event.id`, čo umožňuje presnú identifikáciu udalosti pri kliknutí a následnú navigáciu na obrazovku s detailom.

Automatické skrývanie skončených udalostí: `AppViewModel` vystavuje computed property `activeEvents`, ktorá vracia len udalosti, ktorých koniec je v budúcnosti:

```
val activeEvents: List<Event>
    get() = _events.values.filter { event ->
        val end = event.dateTo?.time ?: event.dateFrom?.time
        end == null || end > System.currentTimeMillis()
    }

```

Výpis kódu 6: Computed property activeEvents v AppViewModel

Udalosti bez dátumu sú vždy zobrazené. Skončené udalosti zmiznú z UI automaticky bez mazania z databázy — interné `_events` zostáva kompletné pre potreby histórie a kontroly neúčastí.

5.2.4 Okruh pôsobnosti a filtrovanie

Markery mimo nastaveného okruhu od používateľa sú automaticky skryté (`marker.isVisible = false`). Okruh je vizualizovaný ako žltý polopriesvitný kruh, ktorého polomer je prepojený s nastaveniami v `AppViewModel`.

5.2.5 Personalizácia zobrazenia

Používateľ môže prepínať medzi piatimi štýlmi mapového podkladu: *NORMAL, SATELLITE, HYBRID, TERRAIN* a špeciálnym *DARK* režimom využívajúcim vlastnú definíciu štýlu vo formáte JSON.

5.3 Eventy — implementácia

5.3.1 Tvorba eventu

Obrazovka tvorby eventu (`EventCreationScreen.kt`) sa otvorí po potvrdení polohy na mape. Formulár obsahuje nasledujúce polia:

- **Názov a popis** — textové polia, obe sú povinné.
- **Miesto** — predvyplnené adresou získanou z Geocoderu / Places API.
- **Cena a počet účastníkov** — hodnota 0 znamená zadarmo, resp. neobmedzený počet.
- **Dátum, čas a trvanie** — výber cez `DatePickerDialog` a `TimePickerDialog`; hodnota `dateTo` sa vypočíta automaticky z trvania.
- **Kategória a podkategória** — výber z `CATEGORY_MAP`.
- **Viditeľnosť** — verejné alebo súkromné; súkromný event vyžaduje heslo.
- **Typ check-in** — `none`, `qr`, `manual` alebo `qr_and_manual`.
- **Deadline odhlásenia** — počet hodín pred eventom, po uplynutí ktorých sa odhlásenie penalizuje.

Upload obrázkov cez Cloudinary: Obrázky sa vyberajú z galérie pomocou `ActivityResultContra` a uploadujú sa ako `multipart/form-data` priamo na Cloudinary API bez potreby vlastného backendu. Prístupové kľúče (`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_UPLOAD_PRESET`) sú uložené v `local.properties` a sprístupnené cez `BuildConfig`.

Uloženie eventu: Metóda `AppViewModel.saveEventToMap()` uloží event súčasne do in-memory mapy `_events` aj do databázy Firestore:

```
fun saveEventToMap(event: Event) {
    _events[event.id] = event
    viewModelScope.launch {
        db.collection("events")
          .document(event.id)
          .set(event.toMap())
          .await()
    }
}

```

Výpis kódu 7: Uloženie eventu do Firestore

5.3.2 Detail eventu

Obrazovka detailu eventu (`EventDetailScreen.kt`) je rozdelená do troch tabov.

Tab 0 — Detail: Zobrazuje všetky informácie o evente. Kľúčové stavy používateľa sú určené nasledovne:

```
val isCreator    = event.creatorId == currentUserId
val isAttending  = currentUserId in event.attendees
val isOnWaitlist = currentUserId in event.waitlist
val isFull       = event.participants > 0 &&
                   event.attendees.size >= event.participants
val isBanned     = currentUserId in event.bannedUsers

```

Výpis kódu 8: Stavové premenné v detaile eventu

Tlačidlá akcie sa dynamicky menia podľa stavu — tvorca vidí úpravu a mazanie, prihlásený účastník odhlásenie, ostatní prihlásenie alebo zaradenie do poradovníka.

Tab 1 — Účastníci: Zobrazuje zoznam účastníkov s avatarmi. Tvorca môže manuálne označiť príchod účastníka (ak `checkInType == "manual"`) alebo ho odstrániť — odstránený účastník je zároveň zablokovaný pre daný event a prvý z poradovníka je automaticky povýšený.

Tab 2 — Chat: Real-time chat implementovaný pomocou Firestore `snapshotListener`. Prístup majú výlučne prihlásení účastníci a tvorca eventu.

5.3.3 Správa účasti

Správa účasti je implementovaná v `AppViewModel` prostredníctvom troch metód.

joinEvent — prihlásenie na event: Ak je event plný, používateľ je zaradený do poradovníka; inak priamo medzi účastníkov:

```
fun joinEvent(eventId: String, userId: String) {
    val isFull = event.participants > 0 &&
                 event.attendees.size >= event.participants
    val updated = if (isFull)
        event.copy(waitlist  = event.waitlist  + userId)
    else
        event.copy(attendees = event.attendees + userId)
    // uloží do Firestore
}

```

Výpis kódu 9: Prihlásenie na event

leaveEvent — odhlásenie z eventu: Pri odhlásení sa automaticky povýši prvý používateľ z poradovníka. Ak sa používateľ odhlási po uplynutí deadlinu, je mu zaznamenaný no-show:

```
fun leaveEvent(eventId: String, userId: String) {
    val applyPenalty = event.isPastCancelDeadline()
    var updated = event.copy(attendees = event.attendees - userId)
    if (updated.waitlist.isNotEmpty()) {
        val next = updated.waitlist.first()
        updated = updated.copy(
            attendees = updated.attendees + next,
            waitlist  = updated.waitlist.drop(1)
        )
    }
    if (applyPenalty) recordNoShow(userId, eventId)
}

```

Výpis kódu 10: Odhlásenie z eventu

removeAttendee — odstránenie účastníka tvorcom: Funguje rovnako ako odhlásenie, no navyše zaradí odstráneného účastníka do zoznamu `bannedUsers`, čím mu zamedzí opätovné prihlásenie na daný event.

5.3.4 Filtrovanie udalostí

Filtrovanie je implementované ako samostatná servisná trieda `EventFilterService`, ktorá prijíma kritériá zapuzdrené v dátovej triede `FilterCriteria`:

```
data class FilterCriteria(
```

```
    val name          : String? = null,
    val category      : String? = null,
    val subcategory   : String? = null,
    val participants  : Int?    = null,
    val maxPrice      : Double? = null,
    val dateFrom      : Date?   = null,
    val maxDistanceKm : Double? = null,
    val refLat        : Double  = 0.0,
    val refLng        : Double  = 0.0
)

```

Výpis kódu 11: Dátová trieda FilterCriteria

Trieda `EventFilterService` aplikuje kritériá postupne — ak event nevyhovuje čo i len jednému z nich, je vyradený. Príklad jednej podmienky:

```
if (!criteria.name.isNullOrEmpty() &&
    !event.title.lowercase().contains(criteria.name.lowercase()))
    return false

```

Výpis kódu 12: Príklad podmienky vo EventFilterService

Výpočet vzdialenosti — Haversine: Na výpočet vzdialenosti medzi dvoma GPS súradnicami sa používa Haversine vzorec, ktorý zohľadňuje zakrivenie Zeme. Výsledkom je vzdialenosť v kilometroch ($R = 6371$ km):

```
private fun haversineKm(
    lat1: Double, lon1: Double,
    lat2: Double, lon2: Double
): Double {
    val dLat = Math.toRadians(lat2 - lat1)
    val dLon = Math.toRadians(lon2 - lon1)
    val a = sin(dLat / 2).pow(2) +
            cos(Math.toRadians(lat1)) *
            cos(Math.toRadians(lat2)) *
            sin(dLon / 2).pow(2)
    return 6371.0 * 2 * asin(sqrt(a))
}

```

Výpis kódu 13: Haversine vzorec

Filter bottom sheet — UI: Filter sa zobrazí ako `ModalBottomSheet` po kliknutí na „Nájsť udalosť". Používateľ filtruje podľa názvu, kategórie, ceny, vzdialenosti a dátumu. Po potvrdení sa zostavia kritériá a aplikujú na markery mapy:

```
val filtered = EventFilterService(criteria).filter(events)
markers.values.forEach { it.isVisible = false }
filtered.forEach { event -> markers[event.id]?.isVisible = true }

```

Výpis kódu 14: Aplikovanie filtrov na mapu

Stav filtra sa ukladá do `AppViewModel.filterState`, čím sa zachováva pri navigácii medzi obrazovkami.

5.4 Hodnotenie udalostí

Systém hodnotenia umožňuje účastníkom ohodnotiť udalosť po jej skončení. Hodnotenie je implementované priamo v `AppViewModel`.

5.4.1 Dátový model:

Hodnotenie je uložené priamo v dokumente eventu v troch poliach:

```
val rating      : Double       = 0.0          // vážený priemer
val totalRatings: Int          = 0            // počet hodnotení
val ratedBy     : List<String> = emptyList() // UIDs ktorí už hodnotili

```

Výpis kódu 15: Polia hodnotenia v dátovom modeli eventu

5.4.2 Logika hodnotenia:

Nový priemer sa vypočíta ako vážený priemer — starý priemer sa vynásobí počtom predchádzajúcich hodnotení, pripočíta sa nové hodnotenie a výsledok sa vydelí novým celkovým počtom. Týmto spôsobom sa zachová presnosť bez nutnosti ukladať každé individuálne hodnotenie:

```
fun rateEvent(eventId: String, userId: String, stars: Int) {
    val event = _events[eventId] ?: return
    if (userId in event.ratedBy) return  // každý môže hodnotiť len raz

    val newTotal  = event.totalRatings + 1
    val newRating = ((event.rating * event.totalRatings) + stars) / newTotal

    val updated = event.copy(
        rating       = newRating,
```

```
10            totalRatings = newTotal,
11            ratedBy      = event.ratedBy + userId
12        )
13        _events[eventId] = updated
14        selectedEvent = updated
15        viewModelScope.launch {
16            db.collection("events").document(eventId).update(mapOf(
17                "rating"       to newRating,
18                "totalRatings" to newTotal,
19                "ratedBy"      to updated.ratedBy
20            )).await()
21        }
22    }
23
```

Výpis kódu 16: Implementácia metódy rateEvent

5.4.3 Pravidlá hodnotenia:

- Hodnotiť môžu len účastníci eventu — nie tvorca ani cudzí používatelia.
- Každý používateľ môže hodnotiť len raz — kontrola prebieha cez zoznam `ratedBy`.
- Hodnotenie je v rozsahu 1 až 5 hviezd.
- Priemerné hodnotenie vstupuje ako jeden z faktorov do odporúčacieho systému s váhou 16 %.

5.4.4 Používateľské rozhranie:

Sekcia hodnotenia sa v `EventDetailScreen` (Tab 0) zobrazí len účastníkovi, ktorý ešte nehodnotil. Aktuálne priemerné hodnotenie a celkový počet hodnotení sú viditeľné pre všetkých používateľov.

5.4.5 Check-in systém

Check-in systém umožňuje organizátorovi overiť fyzickú prítomnosť účastníkov na udalosti. Pri tvorbe eventu si organizátor zvolí jeden zo štyroch typov check-inu, ktoré sú zhrnuté v tabuľke **??**.

Generovanie QR kódu: QR kód sa generuje pomocou knižnice ZXing priamo na zariadení organizátora (`QrCodeScreen.kt`). Obsahom kódu je `eventId` — unikátny identifikátor udalosti z Firestore.

Tabuľka 5: Typy check-inu

Hodnota	Popis
`none`	Check-in sa nevyžaduje
`qr`	Účastník naskenuje QR kód
`manual`	Organizátor manuálne označí prítomnosť
`qr_and_manual`	Kombinácia oboch predchádzajúcich

Skenovanie QR kódu: Skenovanie je implementované v `QrScanScreen.kt` pomocou CameraX na zobrazenie náhľadu kamery a ML Kit Barcode Scanning na rozpoznanie kódu. Obrazovka najprv požiada o oprávnenie `CAMERA`. Stratégia `STRATEGY_KEEP_ONLY_LATEST` zabezpečuje spracovanie len posledného snímku. Po úspešnom naskenovaní sa zavolá `appViewModel.checkInAttendee(eventId, userId)`.

Check-in logika: Metóda `checkInAttendee()` overí, či je používateľ účastníkom eventu a či ešte nebol check-inovaný:

```
fun checkInAttendee(eventId: String, userId: String) {
    val event = _events[eventId] ?: return
    if (userId !in event.attendees) {
        checkInError = "Nie si účastníkom tejto udalosti."
        return
    }
    if (userId in event.checkedIn) {
        checkInError = "Už si zaregistrovaný."
        return
    }
    val updated = event.copy(checkedIn = event.checkedIn + userId)
    _events[eventId] = updated
    viewModelScope.launch {
        db.collection("events").document(eventId)
            .update("checkedIn", updated.checkedIn).await()
        checkInSuccess = true
    }
}

```

Výpis kódu 17: Logika check-inu v AppViewModel

Organizátor môže check-in aj zrušiť volaním `checkOutAttendee()`, ktorá odstráni `userId` zo zoznamu `checkedIn`.

Manuálny check-in: Pri type `manual` alebo `qr_and_manual` vidí organizátor v Tab Účastníci pri každom účastníkovi prepínač ○ / ✓. Kliknutím sa zavolá `checkInAttendee()` alebo `checkOutAttendee()`. Účastník s potvrdenou prítomnosťou má zelenú odznak „✓ Prítomný".

Väzba na penalizačný systém: Ak je `checkInType` ≠ `none` a účastník sa neodhlási ani nezaregistruje check-in, po skončení eventu mu je automaticky zaznamenaný no-show.

5.4.6 Systém penalizácií (No-show)

Penalizačný systém motivuje používateľov dodržiavať záväzky voči udalostiam. Ak sa používateľ prihlási na udalosť s povoleným check-inom, fyzicky sa nezúčastní a ani sa včas neodhlási, systém mu zaznamená no-show.

Dátový model: Penalizácie sú uložené priamo v profile používateľa (`UserProfile`):

```
val noShowTimestamps: List<Long>   = emptyList() // čas každého no-show (epoch ms)
val noShowEventIds  : List<String> = emptyList() // dedup - eventy kde bol no-show

val banned          : Boolean      = false        // true = účet zablokovaný
val banReason       : String       = ""           // "no_show" | "reports" | ""
val banTimestamp    : Long         = 0L           // čas banu v epoch ms

```

Výpis kódu 18: Polia no-show a soft ban v UserProfile

Systém implementuje *soft ban* — pri zablokovaní sa používateľské dáta (história, priatelia, správy) nemažú. Administrátor môže priamo vo Firebase Console vidieť dôvod (`banReason`) a čas (`banTimestamp`) banu a manuálne rozhodnúť o odbanovaní alebo permanentnom vymazaní účtu.

Dva spôsoby vzniku no-show:

1. **Neskoré odhlásenie** — používateľ sa odhlási po uplynutí `cancelDeadlineHours` (podrobnejšie popísané v sekcii správy účasti).

2. **Neúčasť** — systém pri štarte skontroluje všetky minulé eventy, kde bol používateľ prihlásený, ale nebol check-inovaný:

```
fun checkNoShows() {
    for (event in _events.values) {
        if (event.checkInType == "none") continue
        if (uid !in event.attendees)    continue
        if (uid in event.checkedIn)     continue
        if (event.id in profile.noShowEventIds) continue
```

```
            val eventEnd = event.dateTo?.time
                ?: event.dateFrom?.time ?: continue
            if (eventEnd > now) continue
            recordNoShow(uid, event.id)
        }
    }

```

Metóda `checkNoShows()` sa volá automaticky po každom prihlásení v sekvencii `loadUserProfile()` → `checkNoShows()`.

Zaznamenanie no-show: Metóda `recordNoShow()` aktualizuje profil používateľa a v prípade dosiahnutia limitu nastaví príznak zablokovania:

```
suspend fun recordNoShow(userId: String, eventId: String) {
    val now           = System.currentTimeMillis()
    val newTimestamps = profile.noShowTimestamps + now
    val newEventIds   = profile.noShowEventIds   + eventId
    val recentCount   = newTimestamps.count {
        it > now - 30L * 24 * 3_600_000
    }
    val shouldBan = recentCount >= 3

    val updates = mutableMapOf<String, Any>(
        "noShowTimestamps" to newTimestamps,
        "noShowEventIds"   to newEventIds
    )
    if (shouldBan) {
        updates["banned"]       = true
        updates["banReason"]    = "no_show"
        updates["banTimestamp"] = now
        cancelPendingInvitations(userId)
    }

    db.collection("users").document(userId).update(updates).await()
}

```

Výpis kódu 19: Implementácia recordNoShow

Stupňovité varovania: Po zaznamenaní no-show sa používateľovi zobrazí upozornenie podľa aktuálneho počtu, ako je uvedené v tabuľke **??**.

Tabuľka 6: Správanie systému podľa počtu no-show za posledných 30 dní

Počet no-show	Akcia
1	Toast: „Máš 1/3 neúčastí za posledných 30 dní."
2	Toast: „Máš 2/3! Ďalšia ťa zablokuje!"
3	`banned = true` → odhlásenie → „Váš účet bol zablokovaný."

Zablokovanie účtu: Ak `shouldBan == true`, pole `banned` sa nastaví na `true` v databáze Firestore. Pri ďalšom prihlásení `loadUserProfile()` toto pole skontroluje a okamžite odhlási používateľa — rovnaký mechanizmus je popísaný v sekcii autentifikácie.

No-show sa počítajú výlučne za posledných 30 dní — po uplynutí tohto obdobia sa staré záznamy nezapočítavajú, čo umožňuje rehabilitáciu používateľa bez nutnosti manuálneho zásahu administrátora.

5.4.7 Systém nahlasovania

Systém nahlasovania umožňuje používateľom upozorniť na nevhodný obsah — či už udalosť alebo iného používateľa. Pri dosiahnutí stanoveného počtu nahlásení sa obsah automaticky odstráni bez nutnosti zásahu administrátora.

Každé nahlásenie je uložené do Firestore kolekcie `reports` s poliami `reporterId`, `targetId`, `targetType` (`ëvent"` alebo `üser"`), `reason` a `timestamp`.

Dôvody nahlasovania: Zoznam dôvodov a hranica automatickej akcie sú definované ako konštanty v `AppViewModel`:

```
companion object {
    val REPORT_REASONS = listOf(
        "Nevhodný obsah",
        "Spam",
        "Falošná udalosť / profil",
        "Obťažovanie",
        "Iné"
    )
    private const val REPORTS_TO_BAN = 3
}

```

Výpis kódu 20: Konštanty systému nahlasovania

Používateľské rozhranie: Nahlásenie sa vyvolá kliknutím na ikonu príznaku v `EventDetailScreen` (pre udalosť) alebo v `PublicProfileScreen` (pre cudzí profil). Zobrazí sa `AlertDialog` so zoznamom dôvodov ako prepínače (*radio buttons*). Tlačidlo „Nahlásiť"je aktívne len ak je niektorý dôvod vybratý.

Logika nahlasovania udalosti: Metóda `reportEvent()` prebieha v troch krokoch — kontrola duplikátu, uloženie nahlásenia a prípadné automatické vymazanie udalosti:

```
fun reportEvent(reporterId: String, eventId: String, reason: String) {
    // 1. Kontrola duplikátu
    val dup = db.collection("reports")
        .whereEqualTo("reporterId", reporterId)
        .whereEqualTo("targetId",   eventId)
        .whereEqualTo("targetType", "event")
        .get().await()
    if (!dup.isEmpty) {
        reportError = "Túto udalosť si už nahlásil/a."
        return
    }

    // 2. Uloženie nahlásenia
    val docRef = db.collection("reports").document()
    docRef.set(Report(...).toMap()).await()

    // 3. Ak celkový počet nahlásení >= 3, udalosť sa automaticky vymaže
    val total = db.collection("reports")
        .whereEqualTo("targetId",   eventId)
        .whereEqualTo("targetType", "event")
        .get().await()
    if (total.size() >= REPORTS_TO_BAN) deleteEvent(eventId)
}

```

Výpis kódu 21: Implementácia metódy reportEvent

Logika nahlasovania používateľa: Metóda `reportUser()` funguje rovnakým spôsobom. Pri dosiahnutí prahu sa namiesto mazania udalosti nastaví príznak zablokovania na profile používateľa:

```
if (total.size() >= REPORTS_TO_BAN) {
    val banTime = System.currentTimeMillis()
    db.collection("users")
      .document(targetUserId)
      .update(mapOf(
          "banned"       to true,
          "banReason"    to "reports",
```

```
            "banTimestamp" to banTime
        )).await()
    cancelPendingInvitations(targetUserId)
}

```

Výpis kódu 22: Automatické zablokovanie používateľa

Pri každom zablokovaní (no-show aj nahlásenia) sa automaticky vymažú všetky čakajúce pozvánky adresované zabanovanému používateľovi volaním `cancelPendingInvitations()`.

Pravidlá systému: Tabuľka **??** sumarizuje pravidlá platné pre systém nahlasovania.

Tabuľka 7: Pravidlá systému nahlasovania

Pravidlo	**Detail**
Kto môže nahlásiť	Každý prihlásený používateľ okrem tvorcu udalosti a seba samého.
Duplikát	Jeden používateľ môže nahlásiť ten istý cieľ len raz.
Prah	3 unikátne nahlásenia spustia automatickú akciu.
Udalosť	Vymazaná z Firestore aj z in-memory mapy.
Používateľ	`banned = true` → odhlásenie pri ďalšom prihlásení.

5.5 Implementácia komunikačného modulu v reálnom čase

Základom interaktivity je integračná vrstva služby **Firebase Cloud Firestore**, ktorá umožňuje obojsmernú synchronizáciu dát v reálnom čase bez nutnosti vlastnej serverovej infraštruktúry a správy WebSocket spojení.

Správy sú uložené v podkolekcii `events/{eventId}/messages/` s poliami `senderId`, `senderName`, `text` a `timestamp`.

5.5.1 Reaktívna synchronizácia dát

Technické jadro komunikácie tvorí metóda `addSnapshotListener`. Na rozdiel od štandardných HTTP požiadaviek vytvára trvalé spojenie s podkolekciou správ. Akonáhle dôjde k zápisu novej správy, Firebase iniciuje push notifikáciu a aplikácia aktualizuje reaktívny stav `chatMessages` vo ViewModeli.

```
fun startChatListener(eventId: String) {
    chatListener?.remove()
    chatListener = db.collection("events").document(eventId)
        .collection("messages")
        .orderBy("timestamp")
        .addSnapshotListener { snapshot, _ ->
            _chatMessages.clear()
            snapshot?.documents?.forEach { doc ->
                doc.toObject(ChatMessage::class.java)?.let {
                    _chatMessages.add(it)
                }
            }
        }
}

```

Výpis kódu 23: Implementácia real-time listenera vo ViewModeli

5.5.2 Systém priateľstiev

Systém priateľstiev umožňuje používateľom nadviazať spojenie a komunikovať priamo cez správy. Implementácia pokrýva posielanie žiadostí, ich prijímanie a odmietanie, odstraňovanie priateľov a priamu komunikáciu.

Každé priateľstvo je uložené do Firestore kolekcie `friendships` s poliami `requesterId`, `receiverId`, `status` (`"pending"` alebo `äccepted"`) a `timestamp`.

Stavy priateľstva: Pre každú dvojicu používateľov môže existovať práve jeden vzťah. Metóda `getFriendshipStatus()` vráti jeden zo štyroch stavov uvedených v tabuľke **??**:

```
fun getFriendshipStatus(currentUserId: String, otherUserId: String): String {
    val f = _friendships.firstOrNull {
        (it.requesterId == currentUserId && it.receiverId == otherUserId) ||
        (it.requesterId == otherUserId   && it.receiverId == currentUserId)
    } ?: return "none"

    return when {
        f.status == "accepted"         -> "friends"
        f.requesterId == currentUserId -> "pending_sent"
        else                           -> "pending_received"
    }
}

```

Výpis kódu 24: Určenie stavu priateľstva

Tabuľka 8: Stavy priateľstva

Stav	Význam
`none`	Žiadny vzťah medzi používateľmi.
`pending_sent`	Aktuálny používateľ poslal žiadosť.
`pending_received`	Aktuálny používateľ dostal žiadosť.
`friends`	Priateľstvo je akceptované.

Stav sa využíva v `PublicProfileScreen` na zobrazenie správneho tlačidla — „Pridať priateľa", „Žiadosť odoslaná", „Prijať / Odmietnuťalebo „Odstrániť priateľa".

Načítanie priateľstiev: Pri prihlásení sa načítajú všetky dokumenty, kde je používateľ buď odosielateľom alebo prijímateľom. Odmietnuté žiadosti sa nenačítavajú — zabraňuje sa tým opakovanému posielaniu žiadostí od toho istého používateľa:

```
fun loadFriendships(uid: String) {
    val sent     = db.collection("friendships")
        .whereEqualTo("requesterId", uid).get().await()
    val received = db.collection("friendships")
        .whereEqualTo("receiverId",  uid).get().await()
    (sent.documents + received.documents).forEach { doc ->
        val f = Friendship.fromFirestore(doc.id, doc.data!!)
        if (f.status != "declined") _friendships.add(f)
    }
}

```

Výpis kódu 25: Načítanie priateľstiev z Firestore

Odoslanie žiadosti: Pred vytvorením novej žiadosti sa skontroluje, či medzi dvojicou používateľov už vzťah existuje:

```
fun sendFriendRequest(fromId: String, toId: String) {
    val alreadyExists = _friendships.any {
        (it.requesterId == fromId && it.receiverId == toId) ||
        (it.requesterId == toId   && it.receiverId == fromId)
    }
    if (alreadyExists) {
        friendshipError = "Žiadosť o priateľstvo už existuje."
        return
    }
    val friendship = Friendship(
        id          = docRef.id,
        requesterId = fromId,
```

```
        receiverId  = toId,
        status      = "pending",
        timestamp   = System.currentTimeMillis()
    )
    docRef.set(friendship.toMap()).await()
    _friendships.add(friendship)
}

```

Výpis kódu 26: Odoslanie žiadosti o priateľstvo

Odpoveď na žiadosť: Pri prijatí sa stav aktualizuje na `accepted`; pri odmietnutí sa dokument z Firestore aj z lokálnej pamäte vymaže:

```
fun respondToFriendRequest(friendshipId: String, accept: Boolean) {
    if (accept) {
        db.collection("friendships").document(friendshipId)
            .update("status", "accepted").await()
        _friendships[idx] = _friendships[idx].copy(status = "accepted")
    } else {
        db.collection("friendships").document(friendshipId)
            .delete().await()
        _friendships.removeAll { it.id == friendshipId }
    }
}

```

Výpis kódu 27: Prijatie alebo odmietnutie žiadosti

Odstránenie priateľa (`removeFriend()`) funguje rovnakým spôsobom — vyhľadá akceptované priateľstvo a vymaže ho z Firestore aj z lokálnej pamäte.

Priama komunikácia: Priatelia si môžu písať správy cez `FriendChatScreen`. ID konverzácie sa generuje deterministicky zo zoradených UID oboch používateľov — zaručuje sa tak, že obaja vždy pristupujú k tomu istému dokumentu bez ohľadu na to, kto rozhovor začal:

```
private fun directConversationId(uid1: String, uid2: String) =
    if (uid1 < uid2) "${uid1}_${uid2}" else "${uid2}_${uid1}"

```

Výpis kódu 28: Deterministické generovanie ID konverzácie

Správy sa ukladajú do Firestore kolekcie `directMessages/{conversationId}/messages` a synchronizujú sa v reálnom čase cez `snapshotListener` — rovnaký mechanizmus ako chat v udalostiach.

Používateľské rozhranie: Obrazovka `FriendsScreen` je dostupná z bočného menu pod položkou „Priateliaä je rozdelená do dvoch sekcií:

- **Žiadosti o priateľstvo** — zobrazujú sa len ak existujú, s tlačidlami „Prijaťä „Odmietnuť".
- **Priatelia** — zoznam akceptovaných priateľov s možnosťou otvoriť priamu konverzáciu alebo priateľa odstrániť.

5.6 Odporúčací systém

Pre aplikáciu Joinly sme zvolili **Weighted Hybrid System** — váhový hybridný systém kombinujúci tri prístupy:

- **Content-Based** — odporúčania na základe kategórií navštívených eventov,
- **Kontextový** — zohľadňuje vzdialenosť, čas a popularitu,
- **Explicitná spätná väzba** — bonus za eventy z obľúbených kategórií.

Každý z týchto prístupov prispieva do výsledného skóre s nastaviteľnou váhou, ktorú si používateľ môže upraviť priamo v aplikácii pomocou sliderov.

Prečo hybridný systém?

1. **Kvalitné metadáta** — každý event má kategóriu, podkategóriu, lokáciu a hodnotenie.
2. **Menšia používateľská báza** — Collaborative filtering vyžaduje veľké množstvo používateľov; hybridný prístup funguje aj bez nich.
3. **Vysvetliteľnosť** — systém dokáže zdôvodniť odporúčanie (napr. „odporúčame ti toto, lebo máš rád športové eventy").
4. **Flexibilita** — používateľ si sám nastaví čo je pre neho dôležitejšie (blízkosť vs. kategória vs. popularita).
5. **Real-time výpočet** — skóre sa počíta za behu, nevyžadujú sa predpočítané modely.

Dvojúrovňové kategórie: Štandardný Content-Based systém používa jednu úroveň kategórií. V našom systéme sú kategórie dvojúrovňové:

1. **Hlavná kategória** — široká skupina (napr. *Sports*),
2. **Podkategória** — špecifická (napr. *Cycling sports*).

Toto umožňuje presnejšie odporúčania — používateľ, ktorý navštívil cyklistický event, dostane bonus pre ďalšie cyklistické eventy, nie len všeobecne športové. Previazanie medzi úrovňami zabezpečuje trieda **CategoryMapper**.

5.6.1 Váhy a výsledné skóre

Systém používa 6 faktorov, každý s vlastnou váhou. Predvolené hodnoty sú definované v dátovej triede `RecommendationWeights`:

```
data class RecommendationWeights(
    val mainCategory  : Float = 0.20f,  // 20 %
    val subCategory   : Float = 0.16f,  // 16 %
    val distance      : Float = 0.16f,  // 16 %
    val rating        : Float = 0.16f,  // 16 %
    val popularity    : Float = 0.16f,  // 16 %
    val favoriteBonus : Float = 0.16f   // 16 %
)

```

Výpis kódu 29: Predvolené váhy odporúčacieho systému

Hlavná kategória má mierne vyššiu váhu (20 %) pretože je najspoľahlivejším indikátorom záujmu používateľa.

Normalizácia váh: Keďže súčet váh musí byť vždy rovný 1 (100 %), systém hodnoty automaticky normalizuje po každej zmene slidera:

```
fun normalized(): RecommendationWeights {
    val sum = mainCategory + subCategory + distance +
              rating + popularity + favoriteBonus
    if (sum == 0f) return RecommendationWeights()
    return RecommendationWeights(
        mainCategory  = mainCategory  / sum,
        subCategory   = subCategory   / sum,
        distance      = distance      / sum,
        rating        = rating        / sum,
        popularity    = popularity    / sum,
        favoriteBonus = favoriteBonus / sum
    )
```

```
}

```

Výpis kódu 30: Normalizácia váh

$$\sum_{i=1}^{6} w_i = 1{,}0 \quad (\text{vždy } 100\,\%) \tag{1}$$

Vzorec celkového skóre:

$$\boxed{Score = w_1 \cdot C_{main} + w_2 \cdot C_{sub} + w_3 \cdot D + w_4 \cdot R + w_5 \cdot P + w_6 \cdot B_{fav}} \tag{2}$$

Kde:

- $w_1 \ldots w_6$ — nastaviteľné váhy (súčet = 1,0),
- C_{main} — skóre hlavnej kategórie (0,0 – 1,0),
- C_{sub} — skóre podkategórie (0,0 – 1,0),
- D — skóre vzdialenosti (0,0 – 1,0),
- R — skóre ratingu (0,0 – 1,0),
- P — skóre popularity (0,0 – 1,0),
- B_{fav} — bonus za obľúbenú kategóriu (0,0 – 1,0).

Keďže každý faktor je v rozsahu 0,0 – 1,0 a súčet váh je vždy 1,0, výsledné skóre je garantovane v rozsahu:

$$0{,}0 \leq Score \leq 1{,}0 \tag{3}$$

5.6.2 Jednotlivé faktory

Hlavná kategória (C_{main}): Systém zostaví profil používateľa z navštívených eventov — koľko percent navštívených eventov patrilo do ktorej kategórie. Skóre sa vypočíta nasledovne:

```
var mainMatch = userProfile[mainCategory] ?: 0.0
if (userProfile.isEmpty()) mainMatch = 0.5   // novy pouzivatel
else if (mainMatch == 0.0) mainMatch = 0.05  // nikdy nenavstivena

val mainScore = mainMatch * w.mainCategory

```

Výpis kódu 31: Výpočet skóre hlavnej kategórie

- Ak používateľ navštívil 10 eventov, z toho 4 boli „Sports"→ preferencia Sports = 40 % ($C_{main} = 0{,}4$).
- Pre nových používateľov (bez histórie) → $C_{main} = 0{,}5$ (neutrálne skóre).
- Pre nikdy nenavštívenú kategóriu → $C_{main} = 0{,}05$ (aby systém nevrátil prázdny zoznam).

Podkategória (C_{sub}): Funguje rovnako ako hlavná kategória, ale na úrovni podkategórií. Skóre podkategórie dopĺňa hlavnú kategóriu a umožňuje rozlišovať medzi napríklad cyklistickými a plaveckými eventmi v rámci rovnakej skupiny Sports.

Vzdialenosť (D): Vzdialenosť sa počíta Haversinovým vzorcom (popísaným v sekcii filtrovania) a normalizuje sa tak, aby bližšie eventy dostávali vyššie skóre:

$$D = \frac{1}{1 + \frac{distance}{10}} \tag{4}$$

```
val distance  = haversineKm(userLat, userLng,
                            event.latitude, event.longitude)
val distScore = (1.0 / (1.0 + distance / 10.0)) * w.distance

```

Výpis kódu 32: Výpočet skóre vzdialenosti

Príklady normalizovaných hodnôt:

- 0 km → skóre 1,0
- 5 km → skóre 0,67
- 10 km → skóre 0,5
- 20 km → skóre 0,33

Rating (R): Hodnotenie eventu sa normalizuje do rozsahu 0,0 – 1,0. Eventy bez hodnotení dostanú neutrálne skóre 0,5, čím sa zabraňuje ich nespravodlivému penalizovaniu:

```
val ratingScore = if (event.totalRatings > 0) {
    (event.rating / 5.0) * w.rating
} else {
    0.5 * w.rating  // novy event bez hodnoteni = neutralne skore
}

```

Výpis kódu 33: Výpočet skóre ratingu

Popularita (P): Popularita sa meria ako pomer aktuálneho počtu účastníkov voči maximálnej kapacite eventu. Ak event nemá stanovený limit, používa sa strop 100:

```
val maxCap   = if (event.participants > 0)
    event.participants.toDouble() else 100.0
val popScore = min(event.attendees.size.toDouble() / maxCap, 1.0) *
    w.popularity

```

Výpis kódu 34: Výpočet skóre popularity

Bonus za obľúbenú kategóriu (B_{fav}): Obľúbené eventy sa **nepočítajú** do profilu preferencií — používajú sa **výhradne** na tento bonus. To umožňuje používateľovi explicitne označiť čo sa mu páči bez ovplyvnenia celkového profilu navštívených eventov:

```
val favoriteSubCategories = allEvents
    .filter { it.id in favoriteEventIds }
    .map { it.category }
    .toSet()

val favBonus = if (favoriteSubCategories.contains(subCategory))
    w.favoriteBonus.toDouble() else 0.0

```

Výpis kódu 35: Výpočet bonusu za obľúbenú kategóriu

5.6.3 Výstup a používateľské rozhranie

Výsledkom výpočtu je zoznam objektov `ScoredEvent` zoradený zostupne podľa skóre. Odporúčania sa zobrazujú na obrazovke `RecommendedEventsScreen`, kde každý event zobrazuje aj percentuálne skóre zhody. Používateľ si môže jednotlivé váhy faktorov upraviť pomocou sliderov priamo na tejto obrazovke — po každej zmene sa odporúčania prepočítajú v reálnom čase.

5.7 Implementácia systému pozvánok

Systém pozvánok umožňuje organizátorovi udalosti prizvať konkrétnych používateľov priamo cez ich e-mailovú adresu. Pozvaný používateľ dostane pozvánku v zozname *Pozvánky*, kde ju môže prijať alebo odmietnuť.

Každá pozvánka je uložená ako dokument vo Firestore kolekcii `invitations` s poliami `eventId`, `fromUserId`, `toUserId`, `toEmail`, `status` (`"pending"` | `äccepted"` | `"declined"`) a `createdAt`.

Odoslanie pozvánky: Organizátor môže odoslať pozvánku priamo z detailu udalosti kliknutím na ikonu `PersonAdd` v hornej lište. Zobrazí sa dialóg s textovým poľom pre zadanie e-mailovej adresy pozývaného používateľa.

Logika odosielania je implementovaná v metóde `sendInvitation` v `AppViewModel`. Metóda najprv vyhľadá používateľa podľa e-mailovej adresy v kolekcii `users`. Ak používateľ neexistuje, nastaví sa chybová správa. Následne sa overí, či pre danú kombináciu `eventId` a `toUserId` pozvánka už existuje, čím sa zabraňuje duplicitným pozvánkam:

```
fun sendInvitation(
    event        : Event,
    fromUserId   : String,
    fromUserName : String,
    toEmail      : String
) {
    viewModelScope.launch {
        val userSnap = db.collection("users")
            .whereEqualTo("email", toEmail).get().await()
        if (userSnap.isEmpty) {
            invitationError = "Pouzivatel s tymto e-mailom neexistuje."
            return@launch
        }
        val toUserId = userSnap.documents.first().id

        val existing = db.collection("invitations")
            .whereEqualTo("eventId",  event.id)
            .whereEqualTo("toUserId", toUserId)
            .get().await()
        if (!existing.isEmpty) {
            invitationError = "Tento pouzivatel uz bol pozvany."
            return@launch
        }

        val docRef     = db.collection("invitations").document()
        val invitation = Invitation(
            id           = docRef.id,
            eventId      = event.id,
            eventTitle   = event.title,
            eventPlace   = event.place,
            fromUserId   = fromUserId,
            fromUserName = fromUserName,
            toUserId     = toUserId,
            toEmail      = toEmail
        )
        docRef.set(invitation.toMap()).await()
```

```
            invitationSuccess = "Pozvanka bola odoslana."
        }
    }

```

Výpis kódu 36: Odoslanie pozvánky s kontrolou duplicity

Načítanie a zobrazenie pozvánok: Pozvánky sa načítavajú pri spustení aplikácie v metóde `loadInvitations`, ktorá z Firestore načíta všetky dokumenty, kde `toUserId` zodpovedá aktuálne prihlásenému používateľovi.

Obrazovka `MyInvitationsScreen` rozdeľuje pozvánky do dvoch skupín: *Čakajúce* (stav `pending`) a *Vybavené* (stav `accepted` alebo `declined`). Každá pozvánka je zobrazená ako karta s názvom udalosti, miestom konania, menom odosielateľa a farebným stavovým odznačkom.

Reakcia na pozvánku: Prijatie alebo odmietnutie pozvánky rieši metóda `respondToInvitation`. V prípade prijatia sa stav aktualizuje na `accepted` a zároveň sa automaticky zavolá `joinEvent`, čím sa používateľ priradí medzi účastníkov udalosti. Pri odmietnutí sa stav nastaví iba na `declined` bez ďalšej akcie:

```
fun respondToInvitation(
    invitation    : Invitation,
    accept        : Boolean,
    currentUserId : String
) {
    val newStatus = if (accept) "accepted" else "declined"
    val idx = _invitations.indexOfFirst { it.id == invitation.id }
    if (idx >= 0) _invitations[idx] = invitation.copy(status = newStatus)

    viewModelScope.launch {
        db.collection("invitations").document(invitation.id)
            .update("status", newStatus).await()
        if (accept) joinEvent(invitation.eventId, currentUserId)
    }
}

```

Výpis kódu 37: Reakcia na pozvánku

Výsledok operácie je používateľovi oznámený prostredníctvom správy `Snackbar`, ktorú riadia stavové premenné `invitationSuccess` a `invitationError` v `AppViewModel`.

5.8 Implementácia systému lokálnych notifikácií

Aplikácia Joinly informuje používateľov o nadchádzajúcich udalostiach prostredníctvom lokálnych notifikácií systému Android. Na rozdiel od push notifikácií, ktoré vyžadujú serverovú infraštruktúru, lokálne notifikácie sú naplánované priamo na zariadení a nevyžadujú aktívne internetové pripojenie v momente doručenia.

Architektúra systému: Systém pozostáva z troch spolupracujúcich komponentov:

- **NotificationHelper** — singleton objekt zodpovedný za plánovanie a rušenie notifikácií,
- **NotificationReceiver** — prijímač systémových správ, ktorý zostavuje a zobrazuje notifikáciu,
- **Notification** — dátový model zabezpečujúci perzistenciu naplánovaných notifikácií vo Firestore.

Notifikačný kanál a správa oprávnení: Od verzie Android 8.0 (Oreo) je povinné zaregistrovať notifikačný kanál pred prvým použitím. Kanál s identifikátorom `joinly_events` a prioritou `IMPORTANCE_HIGH` sa vytvára pri štarte aplikácie. Od verzie Android 13 (Tiramisu) vyžaduje zobrazovanie notifikácií explicitné oprávnenie `POST_NOTIFICATIONS`, ktoré aplikácia vyžiada pri prvom spustení. Oprávnenie `SCHEDULE_EXACT_ALARM` je deklarované v `AndroidManifest.xml`.

Plánovanie notifikácie: Notifikácia sa naplánuje automaticky v momente, keď sa používateľ pripojí k udalosti ako potvrdený účastník. Upozornenie sa odošle hodinu pred začiatkom udalosti. Plánovanie zabezpečuje `AlarmManager` prostredníctvom metódy `setExactAndAllowWhileIdle`, ktorá garantuje spustenie aj v prípade, že je zariadenie v úspornom režime (*Doze mode*). Na zariadeniach s Android 12+ sa pred použitím presného alarmu overí dostupnosť oprávnenia:

```
fun scheduleEventNotification(context: Context, event: Event) {
    val dateFrom = event.dateFrom ?: return
    val notifyAt = dateFrom.time - NOTIFY_BEFORE_MS   // 1 hodina pred
    if (notifyAt <= System.currentTimeMillis()) return

    val alarmManager = context.getSystemService(AlarmManager::class.java)
    val pending = buildPendingIntent(context, event)

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S
            && !alarmManager.canScheduleExactAlarms()) {
        alarmManager.setAndAllowWhileIdle(
            AlarmManager.RTC_WAKEUP, notifyAt, pending
```

```
13            )
14        } else {
15            alarmManager.setExactAndAllowWhileIdle(
16                AlarmManager.RTC_WAKEUP, notifyAt, pending
17            )
18        }
19    }
20
```

Výpis kódu 38: Plánovanie notifikácie pomocou AlarmManager

Každý alarm je identifikovaný pomocou `PendingIntent` s metadátami udalosti. Ako kľúč pre `requestCode` sa používa `eventId.hashCode()`, čo zaručuje jednoznačnú identifikáciu každej naplánovanej notifikácie.

Zobrazenie notifikácie: Keď `AlarmManager` spustí alarm, systém doručí broadcast triede `NotificationReceiver`, ktorá implementuje `BroadcastReceiver`. Prijímač z `Intent` extrahuje metadáta udalosti, zostaví text správy a zobrazí notifikáciu cez `NotificationManagerCompat` s prioritou `PRIORITY_HIGH` a automatickým zatvorením po kliknutí.

Trieda `NotificationReceiver` je registrovaná v súbore `AndroidManifest.xml` ako prijímač systémových správ.

Zrušenie notifikácie: Ak používateľ opustí udalosť alebo ak organizátor udalosť vymaže, naplánovaný alarm sa zruší volaním `cancelEventNotification`. Zrušenie využíva príznak `FLAG_NO_CREATE` — ak alarm neexistuje, metóda nevyvolá chybu:

```
1   fun cancelEventNotification(context: Context, eventId: String) {
2       val alarmManager = context.getSystemService(AlarmManager::class.java)
3       val intent = Intent(context, NotificationReceiver::class.java)
4       val pending = PendingIntent.getBroadcast(
5           context,
6           eventId.hashCode(),
7           intent,
8           PendingIntent.FLAG_NO_CREATE or PendingIntent.FLAG_IMMUTABLE
9       )
10      pending?.let { alarmManager.cancel(it) }
11  }
12
```

Výpis kódu 39: Zrušenie naplánovanej notifikácie

Perzistencia vo Firestore: Okrem lokálneho alarmu sa každá naplánovaná notifikácia uloží aj do Firestore kolekcie `notifications` s poliami `userId`, `eventId`,

`scheduledAt` a príznakom `sent`. Záznam sa pri opustení udalosti alebo jej vymazaní tiež odstráni. Perzistencia vo Firestore slúži ako záloha pre prípad, keby sa lokálny alarm stratil napríklad po reštarte zariadenia.

6 Testovanie výsledkov

6.1 Metodológia a testovacie dáta

Testovanie sme rozdelili na dve časti:

1. **Analýza vplyvu váh:** Sledovanie zmien odporúčaní pri systematickej zmene váhového vektora pre jedného používateľa a overenie citlivosti systému na jednotlivé faktory.

2. **Overenie funkčnosti predpovede:** Overenie, či systém správne predpovedá relevantné udalosti na základe histórie používateľa, a nájdenie optimálnej konfigurácie váh pomocou metriky Precision@5.

6.2 Analýza vplyvu váh

6.2.1 Cieľ

Cieľom tejto časti bolo pochopiť, ako jednotlivé faktory váhového vektora reálne ovplyvňujú výsledné skóre odporúčaní, a identifikovať ktoré faktory sú pre personalizáciu skutočne relevantné.

6.2.2 Testovací používateľ

Na účely testovania používame náhodného syntetického používateľa, ktorého budeme nazývať Janko. Janko navštívil celkovo 8 udalostí. Ich rozdelenie podľa hlavných kategórií je spracované v tabuľke **??**.

Tabuľka 9: Rozdelenie návštev podľa kategórií

Kategória	**Počet návštev**
Sports	4
Education	3
Business & Tech	1
Spolu	**8**

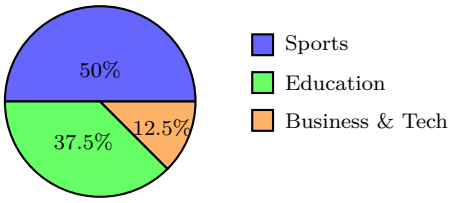
Tabuľka 10: Navštívené udalosti a ich kategorizácia

Podkategória	**Kategória**
Pitch Night	Business & Tech
Workshop (ručné práce)	Education
Osobný rozvoj	Education
Strength & Conditioning	Sports
Team sports	Sports
Winter sports	Sports
Osobný rozvoj	Education
Recreational sports	Sports

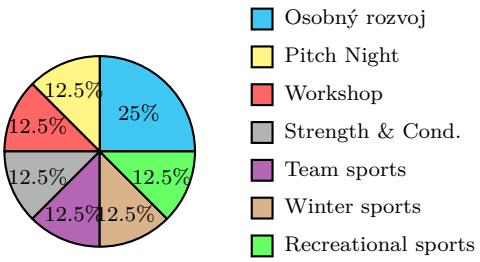
Okrem histórie navštívených podujatí systém berie do úvahy aj udalosti označené ako obľúbené. Tieto majú vyššiu váhu vďaka faktoru B_{fav}. Janko označil ako obľúbenú udalosť v podkategórii *Strength & Conditioning* (Sports).

6.2.3 Vizuálna analýza profilu

Na obrázku **??** vidíme, že zatiaľ čo na úrovni hlavných kategórií dominuje šport (50 %), na úrovni podkategórií sú záujmy veľmi diverzifikované - každá podkategória má zastúpenie len 12,5–25 %.



Obr. 9: Hlavné kategórie



Obr. 10: Podkategórie

Obr. 11: Preferencie používateľa Janka na úrovni kategórií a podkategórií

6.2.4 Výsledky testovania váh

Pre výpočet finálneho skóre používame vážený súčet faktorov (vzťah **??**):

$$Score = w_1 \cdot C_{main} + w_2 \cdot C_{sub} + w_3 \cdot D + w_4 \cdot R + w_5 \cdot P + w_6 \cdot B_{fav} \tag{5}$$

Definovali sme sedem testovacích scenárov so systematickou zmenou váh w_1 až w_6, ktoré sa nachádzajú v priloženom dokumente `dop_system_vysledky.pdf`. Každý test obsahuje konfiguráciu váh, prehľad piatich najlepších (TOP 5) odporúčaní a podrobný

rozbor príspevkov jednotlivých faktorov k výslednému skóre (detailné výstupy sú uvedené v Prílohe **??**).

Výsledky pri rovnomernom rozdelení váh Pri rovnomernom rozdelení ($w_i \approx 16.67\,\%$) dominujú v odporúčaniach udalosti s vysokým ratingom a obľúbené udalosti. Dôvodom sú vlastnosti normalizovaných vstupov:

- **Obľúbený bonus:** Binárna hodnota (0 alebo 1) - automaticky posúva obľúbené udalosti o celú váhovú kategóriu vyššie.
- **Rating:** Väčšina podujatí má hodnotenie blízke 5 hviezdičkám, takže $R \approx 1.0$ - vytvára konštantný silný tlak na skóre.
- **Popularita:** Rastie pri napĺňaní kapacity; plne obsadené podujatia systém vyraďuje.
- **Vzdialenosť:** Stredne silný vplyv závislý od hustoty podujatí v okolí.
- **Hlavná kategória:** Kvôli diverzifikovanej histórii je C_{main} nízke. Pri rovnomernej váhe ho prekrývajú faktory s prirodzene vyššími hodnotami.
- **Podkategória:** Ešte nižší vplyv ako hlavná kategória - C_{sub} je rozptýlené naprieč mnohými podkategóriami, takže pri rovnomernej váhe prakticky nevplýva na výsledné poradie.

Záver: čím viac chceme personalizovať podľa histórie, tým agresívnejšie musíme zvyšovať váhy kategórií.

Faktor	**Reálny vplyv***	**Zdôvodnenie**
Obľúbený bonus	76 %	Binárny charakter (0/1) - najprudšie zmeny v poradí.
Rating	74 %	$R \approx 0.95$ - takmer fixný prírastok ku skóre.
Vzdialenosť	64 %	Vysoký rozptyl medzi blízkymi (0.85) a vzdialenými (0.35).
Hlavná kategória	41 %	$C_{main} = 0.44$ pre šport (4/8 eventov). Závisí od histórie.
Podkategória	30 %	$C_{sub} = 0.11$ (1/8 eventov v každej podkategórii). Nízky vplyv kvôli rozptýlenej histórii.
Popularita	31 %	Najslabší faktor - nízka obsadenosť v testovacej sade.

Tabuľka 11: Vplyv faktorov pri 60 % konfigurácii (*priemerný podiel na skóre TOP 5)

Porovnanie vplyvu faktorov pri dominantnej váhe 60 %

Zhodnotenie faktora podkategórie Podkategória má najnižší reálny vplyv, jej zachovanie v algoritme má však strategický význam:

- **Implicitné učenie:** Systém zachytí zmenu záujmov skôr, než si používateľ manuálne upraví profil.
- **Jemnosť odporúčaní:** Zabraňuje tomu, aby systém odporúčal všetky športové udalosti, keď používateľ preferuje konkrétne len jogu.

6.3 Overenie funkčnosti predpovede

6.3.1 Cieľ

Táto časť má dva ciele. Prvým je overiť, či systém vôbec dokáže správne predpovedať relevantné udalosti na základe histórie - teda či základná funkčnosť funguje. Druhým cieľom je nájsť optimálnu konfiguráciu váh pre hlavnú kategóriu a podkategóriu, keďže z Časti 1 vyplynulo, že práve tieto dva faktory sú relevantné pre personalizáciu.

6.3.2 Overenie základnej funkčnosti

Overovali sme, či systém správne predpovedá udalosti zodpovedajúce dominantnej kategórii používateľa. Vytvorili sme 5 profilov s jasne definovanou dominantnou kategóriou:

- **Sporťák** - 6× Sports, 1× Art & Culture, 1× Food & Drinks
- **Hudobník** - 5× Music & Entertainment, 1× Sports
- **Umelec** - 4× Art & Culture, 1× Music & Entertainment
- **Technik** - 3× Business & Tech, 1× Education
- **Foodie** - 4× Food & Drinks, 1× Community & Social

Predpoveď je úspešná (*PASS*), ak aspoň 3 z TOP 5 odporúčaní patria do dominantnej kategórie používateľa. Pre každý profil sme spustili algoritmus s 10 konfiguráciami váh, kde sme menili pomer hlavná kategória / podkategória od 100 %/0 % po 0 %/100 %:

Konfigurácia	Hlavná kategória	Podkategória
W01	100 %	0 %
W02	90 %	10 %
W03	80 %	20 %
W04	70 %	30 %
W05	60 %	40 %
W06	50 %	50 %
W07	40 %	60 %
W08	30 %	70 %
W09	20 %	80 %
W10	0 %	100 %

Tabuľka 12: Konfigurácie váh pre overenie základnej funkčnosti

Systém úspešne prešiel všetkými konfiguráciami pre všetkých 5 profilov - vo všetkých prípadoch dokázal na základe histórie správne predpovedať relevantné udalosti. Tento výsledok potvrdzuje, že content-based filtrovanie efektívne extrahuje preferencie z histórie a aplikuje ich pri generovaní odporúčaní.

6.3.3 Testovanie citlivosti predpovede

Po potvrdení základnej funkčnosti sme hľadali optimálnu konfiguráciu pomeru hlavná kategória / podkategória. Testovanie prebehlo na 50 syntetických používateľoch s počtom navštívených udalostí od 1 do 50 a datasete 150 náhodných udalostí. Pre každého používateľa sme získali referenčný TOP 5 na základe plnej histórie, následne sme skrývali každý navštívený event a sledovali, či sa TOP 5 zmenil. Zmena TOP 5 po skrytí eventu je pozitívny signál - systém reaguje na aktuálnu históriu a prispôsobuje odporúčania.

Konfigurácia	Hlavná kat.	Podkategória	AVG citlivosť
T01	100 %	0 %	14 %
T02	90 %	10 %	35 %
T03	80 %	20 %	35 %
T04	70 %	30 %	36 %
T05	60 %	40 %	36 %
T06	50 %	50 %	40 %
T07	40 %	60 %	35 %
T08	30 %	70 %	35 %
T09	20 %	80 %	34 %
T10	0 %	100 %	34 %

Tabuľka 13: Výsledky testovania citlivosti - podiel prípadov, kde sa TOP 5 zmenil po skrytí eventu

T01 (100 % hlavná kategória) dosiahla najnižšiu citlivosť 14 % - systém reagoval na zmenu histórie len v 14 % prípadov. Konfigurácia T06 (50 %/50 %) dosiahla maximum 40 %, pričom T10 (100 % podkategória) skončila na 34 %. Kombinácia oboch faktorov v rovnakom pomere teda reaguje na zmeny histórie lepšie ako ktorýkoľvek faktor samostatne.

6.3.4 Obmedzenia metodiky

Leave-one-out metodika má na syntetických dátach niekoľko prirodzených obmedzení. Používatelia s menej ako 5 navštívenými udalosťami predstavujú cold start scenár, kde história po skrytí jednej udalosti nestačí na spoľahlivú personalizáciu. Pri diverzifikovanej histórii môže skrytie udalosti zmeniť dominantnú kategóriu profilu - systém sa správa správne, ale metrika to hodnotí ako miss. Ak používateľ navštívil kategóriu len raz, po jej skrytí klesne C_{main} na nulu - ide o prirodzenú vlastnosť content-based filtra, nie o chybu implementácie.

6.4 Záver testovania

Testovanie potvrdilo, že systém je schopný na základe histórie správne predpovedať relevantné udalosti - vo všetkých 5 profiloch a naprieč všetkými konfiguráciami váh systém úspešne odporučil udalosti zodpovedajúce dominantným záujmom používateľa. Na základe výsledkov testovania citlivosti sme zvolili konfiguráciu T06 (50 %/50 %) ako optimálnu pre produkčné nasadenie. Treba však poznamenať, že táto konfigurácia nemusí byť vhodná v každom prípade - pri používateľovi s históriou obsahujúcou iba

jeden typ udalosti v rámci kategórie môže systém odporúčať správne kategórie, ale nemusí správne predpovedať konkrétny typ podujatia. Kvalita predpovede závisí nielen od nastavenia váh, ale aj od hĺbky a diverzity histórie používateľa.

6.5 Unit testy

6.5.1 Charakteristika unit testov

Unit testy sú automatizované testy, ktoré overujú správnosť izolovaných častí kódu — konkrétnych funkcií alebo tried — bez závislosti na externých systémoch, ako sú Firebase, Google Maps alebo používateľské rozhranie. Každý test funguje na overenom princípe:

1. **Arrange:** Príprava vstupných dát a stavu.
2. **Act:** Vykonanie testovanej funkcie.
3. **Assert:** Porovnanie výsledku s očakávanou hodnotou.

Ak sa výsledok zhoduje s očakávaním, test je vyhodnotený ako **PASSED**. V opačnom prípade zlyhá (**FAILED**) a poskytne informáciu o mieste a príčine chyby.

6.5.2 Umiestnenie a spustenie testov

Všetky unit testy sú umiestnené v module `app/src/test/` a spúšťajú sa príkazom `./gradlew testDebugUnitTest`. Po vykonaní systém vygeneruje HTML správu s detailnými výsledkami.

6.5.3 Pomocná funkcia testEvent

Pre optimalizáciu kódu a elimináciu redundancie pri vytváraní testovacích objektov bola implementovaná pomocná funkcia `testEvent()`. Táto funkcia obsahuje predvolené hodnoty pre všetky povinné parametre triedy `Event`, čo umožňuje v jednotlivých testoch definovať len tie atribúty, ktoré sú pre daný scenár relevantné.

6.5.4 Popis testovacích skupín

Skupina 1: NoShowCountTest Zameriava sa na funkciu `UserProfile.recentNoShowCount()`, ktorá počíta neúčasti používateľa za posledných 30 dní. Táto logika je kritická pre správne fungovanie penalizačného systému (ban po 3 neúčastiach).

Tabuľka 14: Prehľad testov pre NoShowCount

Názov testu	**Vstup (noShowTimestamps)**	**Očak.**	**Účel**
no timestamps returns zero	prázdny zoznam	0	Overenie nového užívateľa.
two recent no-shows	`listOf(now-1s, now-2s)`	2	Základné počítanie.
older than 30 days ignored	`listOf(now - 31 dní)`	0	Overenie časového limitu.
mix of old and recent	`listOf(31d, 1s, 2s)`	2	Filtrovanie starých záznamov.

Skupina 2: CancelDeadlineTest Overuje funkciu `Event.isPastCancelDeadline()`, ktorá určuje, či používateľ ešte môže zrušiť svoju účasť bez penalizácie.

Tabuľka 15: Prehľad testov pre zrušenie účasti

Scenár	**Parametre**	**Výsledok**	**Logika**
checkInType "none"	akýkoľvek čas	`false`	Bez check-inu niet postihu.
deadline set to 0	akýkoľvek čas	`false`	Možnosť zrušiť kedykoľvek.
past deadline	Event o 1h, deadline 3h	`true`	Odhlásenie po termíne.
before deadline	Event o 10h, deadline 2h	`false`	Bezpečné odhlásenie.

Skupina 3: EventFilterTest Testuje `EventFilterService`, ktorý filtruje zoznam podujatí na základe užívateľských preferencií. Pre účely testu sú definované 4 vzorové podujatia (Jazz, Futbal, Yoga, Tech).

- **Filter by name:** Overuje, že vyhľadávanie je *case-insensitive* (napr. „jazz"nájde „Jazz Evening").
- **Filter by category:** Overuje správnosť zaradenia do kategórií (napr. „Sport"vráti Futbal aj Yogu).
- **Filter by maxPrice:** Testuje numerické porovnávanie cien (napr. do 5 € neukáže konferenciu za 20 €).

Skupina 4: ActiveEventsTest Overuje logiku filtrovania aktívnych podujatí pre zobrazenie na mape.

- **Past event:** Podujatie, ktoré už skončilo, sa nesmie vrátiť.
- **Future event:** Budúce podujatia sa musia vrátiť v zozname.
- **No date event:** Podujatia bez dátumu sú považované za trvalo aktívne.

Skupina 5: RecommendationEngineTest Testuje komplexný algoritmus triedy `RecommendationEngine`, ktorý zoraďuje podujatia podľa relevancie.

Tabuľka 16: Logika odporúčacieho algoritmu

Testovaný faktor	**Očakávané správanie**
Vzdialenosť	Bližšie podujatie musí mať vyšší rank ako vzdialené.
Navštívené	Podujatia v `visitedEventIds` sú z výsledkov vylúčené.
Vlastné	Podujatia vytvorené aktuálnym používateľom sa mu neodporúčajú.
Minulosť	Ukončené podujatia sa do odporúčaní nedostanú.

6.5.5 Súhrnný výsledok testovania

Celkovo bolo vykonaných **24 unit testov**.

```
BUILD SUCCESSFUL
24 tests completed, 0 failed
```

Úspešné absolvovanie všetkých testov potvrdilo stabilitu a správnosť kľúčovej biznis logiky aplikácie Joinly, najmä v oblastiach penalizácie používateľov, filtrovania obsahu a personalizovaných odporúčaní

6.6 Manuálne testovanie systému

V tejto časti sú popísané manuálne testovacie scenáre (Test Cases), ktoré overujú funkčnosť aplikácie z pohľadu koncového používateľa. Výsoledky môžete idieť v prílohe **??**

Tabuľka 17: Zoznam manuálnych testovacích scenárov

ID	Scenár	Kroky	Vstup	Očakávaný výsledok
T01	Registrácia — platné údaje	Otvoriť apku → Register	Platný email, heslo 6+ znakov	Odoslanie verifikácie, presmerovanie na Landing.
T02	Registrácia — slabé heslo	Otvoriť apku → Register	Heslo 3 znaky	Chybová hláška o dĺžke hesla.
T03	Registrácia — existujúci email	Otvoriť apku → Register	Email v databáze	Chybová hláška o duplicitnom účte.
T04	Prihlásenie — neoverený email	Login	Neoverený email	Hláška o nutnosti overiť email.
T05	Prihlásenie — správne údaje	Login	Správne údaje	Presmerovanie na mapu.
T06	Prihlásenie — zlé heslo	Login	Nesprávne heslo	Chybová hláška.
T07	Google prihlásenie	Continue with Google	Výber účtu	Presmerovanie na mapu.
T08	Reset hesla	Forgot password	Platný email	Info o odoslaní resetovacieho emailu.
T09	Vytvorenie eventu	Mapa → FAB → pin	Vyplnené polia	Nový marker na mape.
T10	Súkromný event	Visibility: private	Heslo k eventu	Event viditeľný len po zadaní hesla.
T11	Pripojenie cez heslo	Pripojiť sa cez heslo	Správne heslo	Úspešné pridanie do eventu.
T12	Pridanie sa na event	EventDetail → Pridať sa	Verejný event	Meno v zozname účastníkov.
T13	Plný event	EventDetail → Pridať sa	Kapacita naplnená	Zobrazenie pozície v poradovníku.
T14	Odhlásenie včas	EventDetail → Odhlásiť	Pred deadlinom	Odhlásenie bez varovania.

Pokračuje na ďalšej strane...

ID	Scenár	Kroky	Vstup	Očakávaný výsledok
T15	Odhlásenie po termíne	EventDetail → Odhlásiť	Po deadline	Varovný dialóg o penalizácii.
T16	No-show varovanie	Automaticky	2 no-show / 30 dní	Toast varovanie (2/3).
T17	Zablokovanie účtu (no-show)	Automaticky	3 no-show / 30 dní	Odhlásenie a blokácia účtu.
T18	QR check-in (tvorca)	Účastníci → QR ikona	-–	Zobrazenie unikátneho QR kódu.
T19	QR check-in (hosť)	Scan ikona	Scan QR kódu	Badge „Prítomný"pri mene.
T20	Manuálny check-in	Účastníci → toggle	Klik na meno	Zmena stavu na prítomný.
T21	Chat v evente	Tab Chat	Odoslanie správy	Okamžité zobrazenie správy.
T22	Chat (neúčastník)	Tab Chat	Nie je účastník	Blokovanie písania správ.
T23	Pozvánka emailom	Pozvať → email	Registrovaný email	Pozvánka v sekcii používateľa.
T24	Prijatie pozvánky	Drawer → Pozvánky	Klik na „Prijať"	Zaradenie medzi účastníkov.
T25	Nahlásenie eventu	Ikona vlajky	Výber dôvodu	Potvrdenie o nahlásení.
T26	Nahlásenie užívateľa	Profil → vlajka	Výber dôvodu	Potvrdenie o nahlásení.
T27	Odporúčania	Drawer → Odporúčané	-–	Zoznam zoradený podľa AI.
T28	Filter na mape	Filter ikona	Kategória: Šport	Zobrazenie len športových eventov.
T29	Notifikácie	Nastavenia	Výber času	Uloženie preferencií.

Pokračuje na ďalšej strane...

ID	Scenár	Kroky	Vstup	Očakávaný výsledok
T30	Odhlásenie	Drawer → Odhlásiť	Potvrdenie	Presmerovanie na Landing page.
T31	Povinné polia	Uložiť event	Prázdny formulár	Toast s chybovou hláškou.
T32	Chýbajúci dátum	Uložiť event	Bez dátumu	Výzva na výber dátumu a času.
T33	Chýbajúca kategória	Uložiť event	Bez kategórie	Výzva na výber kategórie.
T34	Chýbajúca podkategória	Uložiť event	Bez podkategórie	Výzva na výber podkategórie.
T35	Dátum v minulosti	Výber dátumu	Včerajší deň	Deň nie je možné vybrať.
T36	Zadanie adresy	Pole Miesto	„Hlavné nám"	Autocomplete návrhy.
T37	Nahratie fotky	Pridať fotky	Výber z galérie	Zobrazenie náhľadu.
T38	Filter — podľa názvu	Filter → Názov	„jazz"	Zobrazenie relevantných eventov.
T39	Filter — podľa kategórie	Filter → Kategória	„Šport"	Zobrazia sa len Šport eventy.
T40	Filter — podkategória	Filter → Podkategória	Konkrétna voľba	Zobrazenie podľa podkategórie.
T41	Filter — max. cena	Filter → Max. cena	„10"	Eventy nad 10€ zmiznú.
T42	Filter — vzdialenosť	Filter → Okruh	5 km	Zobrazia sa eventy do 5 km.
T43	Filter — dátum	Filter → Dátum od	Konkrétny deň	Eventy od daného dátumu.
T44	Soft ban užívateľa	3x nahlásenie profilu	Rôzni užívatelia	Dočasné obmedzenie funkcií účtu.
T45	Odstránenie eventu	3x nahlásenie eventu	Rôzni užívatelia	Automatické stiahnutie eventu z mapy.

Pokračuje na ďalšej strane...

ID	Scenár	Kroky	Vstup	Očakávaný výsledok
T46	Použitie rovnakého mena pri registrácii	Registrácia	Existujúce užívatelské jméno	Chybová hláška o duplicitním jménu.

Táto práca sa zaoberala návrhom a vývojom mobilnej aplikácie Joinly, ktorej cieľom bolo uľahčiť objavovanie a správu spoločenských udalostí s využitím moderných technológií pre platformu Android. V úvodnej kapitole boli vysvetlené základné pojmy potrebné na pochopenie problematiky. Následne bol analyzovaný súčasný stav — existujúce riešenia a ich nedostatky, ktoré motivovali vznik tejto aplikácie. Ďalšia kapitola sa venovala použitým technológiám a nástrojom. Na základe týchto poznatkov boli definované funkčné a nefunkčné požiadavky, ktoré tvorili základ pre návrh riešenia. Návrh bol zdokumentovaný prostredníctvom UML diagramov. Následne prebehla samotná implementácia aplikácie a v záverečnej kapitole bolo vykonané testovanie, ktoré overilo správnosť implementovaných funkcionalít.

Výsledkom práce je funkčná aplikácia pokrývajúca základné požiadavky — autentifikáciu používateľov, správu udalostí, systém účasti, odporúčací systém, chat aj moderačné nástroje. Napriek tomu aplikácia v súčasnom stave nie je plne pripravená na masové nasadenie. Využíva bezplatné úrovne troch cloudových poskytovateľov — Firebase, Google Maps a Cloudinary — ktoré sú vhodné pre vývoj a testovanie, no pri väčšom počte používateľov by narazili na limity výkonu aj nákladov. Do budúcnosti by bolo vhodné pre jednotlivé funkcie využiť špecializovaných poskytovateľov optimalizovaných na škálovateľnosť, prípadne vyvinúť vlastné riešenia prispôsobené konkrétnym potrebám aplikácie.

Ďalším obmedzením je výpočet odporúčacieho systému, ktorý prebieha priamo na zariadení používateľa. Pri väčšom množstve dát by bolo vhodnejšie presunúť túto logiku na server formou cloudových funkcií. Aplikácia taktiež prešla len obmedzeným testovaním v kontrolovanom prostredí — dlhodobé testovanie reálnymi používateľmi by odhalilo ďalšie nedostatky, najmä v oblasti odporúčacieho systému, ktorý bez dostatočného množstva reálnych interakcií nie je možné plnohodnotne vyhodnotiť. Do budúcnosti je možné aplikáciu rozšíriť o prémiové služby pre organizátorov, nákup vstupeniek priamo v aplikácii, ďalšie spôsoby overenia identity, ako aj o riešenie legislatívnych požiadaviek v oblasti GDPR a ochrany osobných údajov.

Napriek týmto obmedzeniam predstavuje aplikácia Joinly funkčný základ, ktorý demonštruje reálnu využiteľnosť moderných mobilných technológií pri riešení každodenných potrieb. Aplikácia prináša hodnotu najmä v spojení komunít okolo spoločných

záujmov, zjednodušení organizácie udalostí a personalizovanom objavovaní podujatí prostredníctvom odporúčacieho systému. Týmto spôsobom napĺňa svoj hlavný cieľ — priblížiť ľudí k udalostiam, ktoré sú pre nich relevantné.

Záver

„Táto práca sa zaoberala návrhom a vývojom mobilnej aplikácie Joinly, ktorej cieľom bolo uľahčiť objavovanie a správu spoločenských udalostí s využitím moderných technológií pre platformu Android. V úvodnej kapitole boli vysvetlené základné pojmy potrebné na pochopenie problematiky. Následne bol analyzovaný súčasný stav — existujúce riešenia a ich nedostatky, ktoré motivovali vznik tejto aplikácie. Ďalšia kapitola sa venovala použitým technológiám a nástrojom. Na základe týchto poznatkov boli definované funkčné a nefunkčné požiadavky, ktoré tvorili základ pre návrh riešenia. Návrh bol zdokumentovaný prostredníctvom UML diagramov. Následne prebehla samotná implementácia aplikácie a v záverečnej kapitole bolo vykonané testovanie, ktoré overilo správnosť implementovaných funkcionalít.

Výsledkom práce je funkčná aplikácia pokrývajúca základné požiadavky — autentifikáciu používateľov, správu udalostí, systém účasti, odporúčací systém, chat aj moderačné nástroje. Napriek tomu aplikácia v súčasnom stave nie je plne pripravená na masové nasadenie. Využíva bezplatné úrovne troch cloudových poskytovateľov — Firebase, Google Maps a Cloudinary — ktoré sú vhodné pre vývoj a testovanie, no pri väčšom počte používateľov by narazili na limity výkonu aj nákladov. Do budúcnosti by bolo vhodné pre jednotlivé funkcie využiť špecializovaných poskytovateľov optimalizovaných na škálovateľnosť, prípadne vyvinúť vlastné riešenia prispôsobené konkrétnym potrebám aplikácie.

Ďalším obmedzením je výpočet odporúčacieho systému, ktorý prebieha priamo na zariadení používateľa. Pri väčšom množstve dát by bolo vhodnejšie presunúť túto logiku na server formou cloudových funkcií. Aplikácia taktiež prešla len obmedzeným testovaním v kontrolovanom prostredí — dlhodobé testovanie reálnymi používateľmi by odhalilo ďalšie nedostatky, najmä v oblasti odporúčacieho systému, ktorý bez dostatočného množstva reálnych interakcií nie je možné plnohodnotne vyhodnotiť. Do budúcnosti je možné aplikáciu rozšíriť o prémiové služby pre organizátorov, nákup vstupeniek priamo v aplikácii, ďalšie spôsoby overenia identity, ako aj o riešenie legislatívnych požiadaviek v oblasti GDPR a ochrany osobných údajov.

Napriek týmto obmedzeniam predstavuje aplikácia Joinly funkčný základ, ktorý demonštruje reálnu využiteľnosť moderných mobilných technológií pri riešení každodenných potrieb. Aplikácia prináša hodnotu najmä v spojení

komunít okolo spoločných záujmov, zjednodušení organizácie udalostí a personalizovanom objavovaní podujatí prostredníctvom odporúčacieho systému. Týmto spôsobom napĺňa svoj hlavný cieľ — priblížiť ľudí k udalostiam, ktoré sú pre nich relevantné. "

(ChatGPT 3, 2024)

Poznámka autora: Časť záveru vygenerovala umelá inteligencia na základe analýzy obsahu verzie dokumentu pre práce písané v MS Worde. Konkrétne išlo o aplikáciu ChatGPT 3 dňa 29. 9. 2024. Rukopis umelej inteligencie je z textu cítiť aj napriek tomu, že celá komunikácia vedúca k tejto forme záveru trvala približne 40 minút. Parafrázujúc ľudové príslovie konštatujeme, že umelá inteligencia je možno dobrý sluha, ale zlý pán. Používajme ju s rozumom.

Pripomíname, že takýto spôsob tvorby obsahu je v záverečnej práci, ako aj v iných akademických textoch, neprípustný. Autorské dielo nesmie obsahovať odseky vytvorené generatívnou umelou inteligenciou. Preto sme ho formátovali ako citovaný text.

Literatúra

1. PRISPIEVATELIA WIKIPÉDIE. *Autentifikácia* [online]. 2025. [cit. 2026-04-14]. Dostupné z: https://sk.wikipedia.org/wiki/Autentifik%C3%A1cia.
2. AUTH0. *Refresh Tokens* [online]. [cit. 2026-04-14]. Dostupné z: https://auth0.com/docs/secure/tokens/refresh-tokens.
3. TIWARI, S. An Introduction to QR Code Technology. In: [online]. IEEE, 2016 [cit. 2026-04-14]. Dostupné z: https://ieeexplore.ieee.org/abstract/document/7966807.
4. OLŠAV, M. *Mobilná aplikácia pre skialpinistov so zameraním na sledovanie, analýzu senzorických údajov a bezpečnostné výstrahy* [online]. 2024. [cit. 2026-04-14]. Dostupné z: https://opac.crzp.sk/?fn=detailBiblioFormChild01BU&sid=9A7D98113A4E7D6F09AF71A0EB46&seo=CRZP-detail-kniha. Diplomová práca.
5. PULLUR, R. *Native vs Cross-Platform: App Size and Performance Comparison* [online]. 2025. [cit. 2026-04-14]. Dostupné z: https://medium.com/@riyaspullurofficial/native-vs-cross-platform-app-size-and-performance-comparison-550cb7f918b0.

Použitie nástrojov umelej inteligencie

OpenAI (2025), ChatGPT 4o, Záver, generovanie textu.

OpenAI (2025), ChatGPT 4o, implementácia, návrh a ladenie Kotlin kódu.

Dodatok A: Algoritmus

Algoritmus 1 Vypočítaj $y = x^n$

Require: $n \geq 0 \vee x \neq 0$

Ensure: $y = x^n$

```
y ⇐ 1
if n < 0 then
    X ⇐ 1/x
    N ⇐ −n
else
    X ⇐ x
    N ⇐ n
end if
while N ≠ 0 do
    if N is even then
        X ⇐ X × X
        N ⇐ N/2
    else {N is odd}
        y ⇐ y × X
        N ⇐ N − 1
    end if
end while
```

Dodatok B: Výpis dlhého kódu

Dlhšie kódy, prípadne kompletné programy vypisujeme do dodatkov práce. Nasledujúci program v jazyku Python vygeneruje súbor `thesis.tex`, ktorý je funkčne totožný s hlavným súborom tohto dokumentu. Stačí vyplniť prehľadný formulár pri volaní funkcie `generate_thesis_tex` na konci kódu a získate hlavný súbor vašej práce.

```
def generate_thesis_tex(
    author,
    reg_nr,
    title,
    title_en,
    day,
    month,
    year,
    keywords,
    keywords_en,
    study_programme,
    study_programme_en,
    study_field,
    study_field_en,
    training_workplace,
    training_workplace_en,
    supervisor,
    consultant=None,
    thesis_type="bp",
    language="sk",
):

    tex_class_option = thesis_type if thesis_type in ["bp", "dp"] else "bp"
    tex_language_option = "en" if language == "en" else ""

    if tex_language_option:
        document_class = f"\\documentclass[{tex_class_option},{tex_language_option
    }]{{FEIstyle}}"
    else:
        document_class = f"\\documentclass[{tex_class_option}]{{FEIstyle}}"

    tex_content = f"""
{document_class}

% Author of the thesis
\\FEIauthor{{{author}}}
```

```
% Evidence number
\\FEIregNr{{{reg_nr}}}

% Title
\\FEItitle{{{title}}}
\\FEItitleEn{{{title_en}}}

% Date
\\FEIdate{{{day}}}{{{month}}}{{{year}}}

% Keywords
\\FEIkeywords{{{keywords}}}
\\FEIkeywordsEn{{{keywords_en}}}

% Further details
\\FEIstudyProgramme{{{study_programme}}}
\\FEIstudyProgrammeEn{{{study_programme_en}}}
\\FEIstudyField{{{study_field}}}
\\FEIstudyFieldEn{{{study_field_en}}}
\\FEItrainingWorkplace{{{training_workplace}}}
\\FEItrainingWorkplaceEn{{{training_workplace_en}}}

% Supervisor
\\FEIsupervisor{{{supervisor}}}
"""

    if consultant:
        tex_content += f"\n\\FEIconsultant{{{consultant}}}\n"

    tex_content += f"""

% Bibliography database file
\\bibliography{{includes/bibliography.bib}}

\\begin{{document}}

\\frontmatter
\\FEIpdfInfo
\\FEIcover
\\FEItitlePage
\\includepdf[pages=-]{{includes/assignment.pdf}}
\\FEIthanks{{includes/thanks}}
\\FEIabstract{{includes/abstract}}
\\FEIabstractEn{{includes/abstractEN}}
```

```
\\FEIcontent
\\FEImanualListOfGlossaries{{includes/manual_glossary}}
\\FEIlistOfAlgorithms
\\FEIlistOfListings

\\mainmatter
\\FEIintroduction{{includes/introduction}}
\\FEIcore{{includes/core}}
\\FEIconclusion{{includes/conclusion}}
\\FEIbibliography
\\FEIaiDeclaration{{includes/ai_declaration}}
\\backmatter
\\FEIappendix{{Algoritmus\\label{{att:algorithms}}}}{{includes/attachmentA}}
\\FEIappendix{{Výpis dlhého kódu\\label{{att:listings}}}}{{includes/attachmentB}}
\\FEIappendix{{Slovníček pojmov\\label{{att:dictionary}}}}{{includes/attachmentC}}

\\end{{document}}
"""

    with open("thesis.tex", "w", encoding="utf-8") as file:
        file.write(tex_content)

    print("Súbor thesis.tex bol úspešne vygenerovaný.")


# Testovacie volanie funkcie
generate_thesis_tex(
    author="RNDr. Juraj Chlpík, PhD.",
    reg_nr="FEI-xxxx-xxxx",
    title="Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme
    LaTeX",
    title_en="Extended thesis template at FEI STU in Bratislava in LaTeX system",
    day="31",
    month="12",
    year="2024",
    keywords="záverečná práca, šablóna, LaTeX, formátovanie textu, citácie",
    keywords_en="Final thesis, template, LaTeX, text formatting, citations",
    study_programme="názov študijného programu",
    study_programme_en="Study program in English",
    study_field="názov študijného odboru",
    study_field_en="Field of the study in English",
    training_workplace="Názov školiaceho pracoviska",
    training_workplace_en="Training Work place",
    supervisor="tituly Meno Priezvisko, tituly",
    consultant="",
```

```
    thesis_type="bp",  # Možnosti: "bp", "dp"
    language="sk",  # Možnosti: "sk" (slovenčina - predvolená), "en" (angličtina)
)
```

Výpis kódu 40: Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting`

Dodatok C: Slovníček pojmov

Abstrakt – Krátke zhrnutie hlavných bodov dokumentu, vrátane cieľov, metodológie a výsledkov.

Bod (pt) – Jednotka merania veľkosti písma; 1 bod sa rovná približne 0,352 mm.

Bold – Písmo, ktoré je tučné, používané na zvýraznenie dôležitých častí textu a v nadpisoch.

Citácia – Uvedenie zdroja informácií v texte, ktoré umožňuje identifikáciu pôvodného materiálu.

Font – Typ písma, ktorý sa používa na zobrazenie textu. Rôzne fonty majú odlišný štýl a vzhľad.

Graf – Vizualizácia údajov, ktorá zobrazuje vzťah medzi premennými.

Kapitola – Hlavná časť dokumentu, ktorá sa zaoberá konkrétnou témou a je zvyčajne označená číslom alebo názvom.

Korektúra – Proces revízie a opravy textu s cieľom odstrániť chyby.

Kurzíva – Typ písma, ktorý je sklonený doprava a používa sa na zvýraznenie textu, názvov alebo cudzích výrazov.

Metodológia – Opis metód a techník použitých pri výskume alebo analýze.

Normostrana – Základná jednotka pre meranie dĺžky textu, ktorá zvyčajne obsahuje 1 800 znakov vrátane medzier.

Obrázok – Grafický prvok alebo fotografia, ktorá ilustruje text.

Odsadenie – Vzdialenosť prvého riadku odseku od okraja stránky.

Odsek – Základná jednotka textu, ktorá obsahuje súvislé myšlienky. V LaTeX-u sa odseky oddeľujú prázdnym riadkom v zdrojovom texte.

Plagiátorstvo – Prezentovanie cudzej práce alebo myšlienok ako vlastných bez riadneho citovania.

Rez písma – Variant typu písma, ktorý určuje štýlové úpravy, ako sú normálny, tučný (bold), kurzíva (italic) alebo podčiarknutie.

Riadkovanie – Vzdialenosť medzi jednotlivými riadkami textu.

Sadzba – Proces usporiadania textu a grafiky na stránke.

Súbor – Elektronický dokument, ktorý obsahuje text alebo dáta.

Šablóna – Prednastavené formátovanie pre rôzne typy textu, ktoré zabezpečuje jednotný vzhľad.

Tabuľka – Usporiadanie údajov do riadkov a stĺpcov.

Titulný list – Prvá stránka dokumentu, ktorá obsahuje základné informácie, ako je názov práce a meno autora.

Typografia – Umenie a technika usporiadania textu tak, aby bol esteticky príťažlivý a čitateľný.

Úvod – Prvá časť dokumentu, ktorá predstavuje tému a ciele výskumu.

Vedecká práca – Akademický dokument, ktorý prezentuje výsledky výskumu, analýzy alebo diskusie o špecifickej téme.

Zdroje primárne a sekundárne – Primárne zdroje sú originálne dokumenty; sekundárne zdroje interpretujú alebo analyzujú primárne zdroje.